Model Augmentation for a Dual-Gantry High-Precision Motion System

Dirk van den Berg
Control Systems Group, Dept. of Electrical Engineering
Eindhoven University of Technology
Eindhoven, The Netherlands

***Abstract*—**

- **Problem: improve settling prediction for a position-dependent dual gantry without surrendering the meaning of its physical parameters**
- **Method: combine a quasi-LPV first-principles baseline, dynamic parallel LFR augmentation, closed-loop rollout training, and state-level orthogonal projection**
- **Evidence: compare baseline, unconstrained augmentation, orthogonal variants, and a black-box model on the completed validation ladder, including relevant motion and settling where available**
- **Main result: report predictive improvement together with the measured limits from short-horizon training, inactive added states, and non-orthogonal true missing dynamics**
- **Conclusion: state when the proposed decomposition is useful and which conditions prevent interpreting it as true parameter recovery**

[TODO: Write the numerical abstract only after the final simulation and Telica result tables are frozen.]

***Index Terms*—**

I. Introduction

ASMPT develops semiconductor assembly and packaging equipment, including high-precision motion systems for advanced packaging, which combines multiple semiconductor functions within a single package. Such technologies place strict demands on alignment accuracy, yield, and manufacturing cost [1], [2].

This paper focuses on a dual-gantry motion system in which two parallel linear actuators support a common beam carrying the payload. The shared beam couples the actuator dynamics, so motion of one actuator influences the other, and the system exhibits position-dependent behaviour [3], [4] and flexible dynamics [3]. In such systems, settling behaviour is a key performance metric. After each motion, the system must wait for residual oscillations to decay before the next placement can begin [5], so settling time directly affects throughput. Furthermore, cross-coupling means that the motion of one gantry can disturb the other, making accurate prediction relevant not only for settling but also for tracking behaviour during simultaneous motion.

Predicting this behaviour therefore requires an accurate representation of the underlying plant dynamics. Such a model can be used as a digital twin, for example to evaluate a changed controller before it is tested on the machine. Digital twins can support offline simulation of semiconductor manufacturing equipment and help optimize operating parameters [6]. Such models are also valuable for control design and for process and root cause analysis. This requires a model with physically meaningful parameters, because a change in machine behaviour can then be traced back to a physical cause, such as a change in stiffness or damping.

White-box models are attractive because they reflect the physics of the system, which makes them interpretable. First-principles (FP) models are a specific type of white-box model, derived from physical laws and described by quantities such as mass, stiffness, and damping. However, purely physics-based models do not always capture all behaviour needed for accurate prediction across the operating range [7]. More accurate FP models can be developed, but this is labour-intensive [8].

Purely data-driven (black-box) models such as neural networks or Gaussian processes can describe measured behaviour well, but they give less physical insight and are often harder to trust at operating conditions not covered by the training data [7]. Typically these models also require large amounts of data to train [7], [9]. In addition, these models need to learn behaviour that is already known from first principles, such as the rigid-body dynamics [8]. Physics-informed approaches, including physics-informed neural networks (PINNs), enforce physical laws as constraints during training [9], [10]. However, these methods require the physics equations to be known and still rely on a black-box model for the entire system [8]. For the plant model of the dual-gantry system, both predictive accuracy and physical interpretability are required. Physical interpretability means that the estimated physical parameters are unique [11]. [**TODO:** Leave out?: Section IV-A gives the conditions under which they equal their true values.] However, neither a purely physics-based nor a purely data-driven model is sufficient on its own [7]. Grey-box modeling is the logical choice, because it is able to combine a physically meaningful baseline model needed for interpretation with data-driven flexibility needed for accurate prediction [12]. The following grey-box approaches appear in practice: parameter re-estimation within the physics model structure, static residual correction on the model output, and dynamic augmentation that extends the baseline with learned states [7], [8]. Model augmentation combines a physics-based baseline model with a learned component, so that the learned component only needs to capture the dynamics the physics-based model cannot describe [8]. However, related work on an ASMPT wire bonder shows that preserving physical interpretability during model updating

remains an open challenge [13].

Learning-based hybrid modeling has already been demonstrated on real industrial systems [14]. However, a key gap remains: there is no grey-box modeling framework for the dual-gantry system that simultaneously addresses coupled multi-input multi-output (MIMO) dynamics, position-dependent behaviour, closed-loop data, and preserves physical interpretability. Model augmentation using a linear fractional representation (LFR) has already been demonstrated, but not for position-dependent baselines or closed-loop data [8]. The related work at ASMPT uses closed-loop data, but keeps the physical parameters fixed during training [13]. Orthogonal-by-construction augmentation is able to preserve the physical meaning of the baseline parameters, but has so far only been formulated for discrete-time input-output models [11]. This paper therefore aims to develop and evaluate such a framework in simulation, so that the plant model becomes more accurate across operating points and motion profiles while remaining physically meaningful.

This paper addresses the following research question: *For the dual-gantry system, how should a physics-based plant model be augmented to capture unmodelled dynamics, improving prediction accuracy across operating points and motion profiles while preserving the physical interpretability of its parameters?* To answer this research question, the following four aspects are investigated:

1) In what form should the baseline model be represented for subsequent augmentation, and does a position-dependent (LPV) formulation improve prediction compared to position-independent (LTI) alternatives?
2) What augmentation structure can effectively capture dynamics not modeled in the physics-based baseline model?
3) How can parameter interpretability be maintained during data-driven augmentation, compared to unconstrained approaches?
4) How well does the augmented model generalize to (i) held-out operating points and (ii) unseen motion profiles, and how does its prediction accuracy compare to a black-box state-space model?

[**TODO:** Confirm whether references and appendices are excluded from the 12-page IEEE limit and whether TU/e requires front matter beyond the IEEE paper.]

II. Dual-Drive Gantry System and Physics Baseline

A. System and first-principles baseline

[**TODO:** state we leave out coulomb friction and that we will use for the experiment in the simulation in section V] The physics baseline of the gantry system used in this paper is the lumped-parameter model of the dual-drive gantry of García-Herreros et al. [3]. The gantry has two actuators that are connected with a cross-arm and a third actuator that moves the payload along this arm (Fig. 1). [**TODO:** Does this make sense to put here?: Dynamics not represented by the baseline are treated as model discrepancy and addressed by the augmentation in Section III.] Define the actuator forces $u_{\text{act}} = [F_{X1}\ F_{X2}\ F_Y]^\top$, the measured positions $q_{\text{act}} = [X_1\ X_2\ Y]^\top$, and the generalized coordinates $q = [X\ \Theta\ Y]^\top$. Let L_b denote the cross-arm length between the two flexible joints. The coordinate and force transformations are

$$q = P^{-\top} q_{\text{act}}, \qquad u = P u_{\text{act}}, \\ P = \begin{bmatrix} 1 & 1 & 0 \\ L_b/2 & -L_b/2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

where u denotes the generalized force. The generalized coordinates are defined as $X = (X_1 + X_2)/2$ and $\Theta = (X_1 - X_2)/L_b$, while Y is unchanged. These coordinates avoid [**TODO:** This needs to make clear why we choose for this realization. a closed kinematic-chain description and make the coupling caused by the nonuniform load distribution explicit] [3]. [**TODO:** How to connect, nonunfirom ... moving payload position] Moving the payload position Y changes the mass distribution of the gantry and therefore its inertia. This position dependence is retained by representing the baseline in linear parameter-varying (LPV) form, with the measured payload position Y as scheduling variable [15].

Following García-Herreros et al. [3], the baseline uses $\cos\Theta \approx 1$ and $\sin\Theta \approx 0$ and assumes that the Coriolis and centripetal force term is negligible compared with the inertia, damping, and stiffness forces. Under these assumptions, the baseline equations of motion are

$$M(Y)\ddot{q}(t) + C\dot{q}(t) + Kq(t) = P u_{\text{act}}(t), \tag{2}$$

with

$$M(Y) = \begin{bmatrix} m_1 + m_2 + m_b + m_h & \frac{L_b}{2}(m_1 - m_2) - m_h Y & 0 \\ \frac{L_b}{2}(m_1 - m_2) - m_h Y & J_b + J_h + \frac{L_b^2}{4}(m_1 + m_2) + m_h(d^2 + Y^2) & -m_h d \\ 0 & -m_h d & m_h \end{bmatrix}, \tag{3}$$

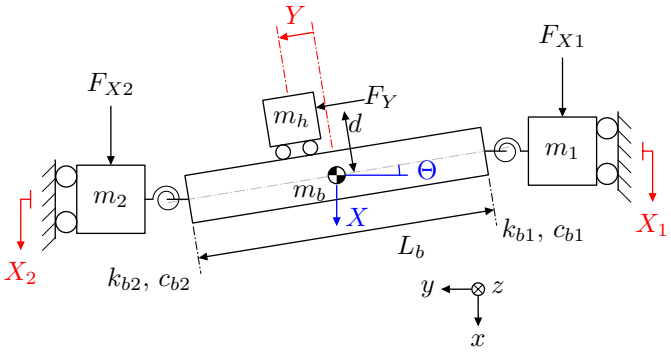


Fig. 1: Still need to add springs to the current torsion. [**TODO:** Lumped-parameter gantry schematic adapted from the mechanical arrangement in [3]. Red labels denote actuator coordinates and blue labels generalized coordinates.]

[**TODO:** Will need to add springs to the current torsion; feedback from Jasper]

$$C = \begin{bmatrix} c_{g1}+c_{g2} & \frac{L_b}{2}(c_{g1}-c_{g2}) & 0 \\ \frac{L_b}{2}(c_{g1}-c_{g2}) & c_{b1}+c_{b2}+\frac{L_b^2}{4}(c_{g1}+c_{g2}) & 0 \\ 0 & 0 & c_y \end{bmatrix},$$
$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & k_{b1}+k_{b2} & 0 \\ 0 & 0 & 0 \end{bmatrix}. \quad (4)$$

[**TODO:** Proposed insertion: "García-Herreros et al. also model Coulomb friction at the three actuators [3, Sec. 2.2]. The baseline omits these terms, so that friction remains a physical effect the augmentation must capture (Section V)." Reason documented in D-204. Second reason? A sign term is not twice differentiable, which [8, Cond. 8] assumes for well-posedness. because: ?] [**TODO:** Look at this section, the above state a lot, look fine] The masses, flexible-joint stiffness and damping coefficients, and [**TODO:** geometric dimensions] are shown in Fig. 1. Here, J_b and J_h denote the yaw inertias, while c_{g1}, c_{g2}, and c_y denote the actuator damping coefficients. The nominal values are taken from [3, Table I] and listed in Appendix A-C [**TODO:** Not sure if to add this, depends on real data or not; they were not identified on the present Telica machine.] To prevent the definition of q from being parameter-dependent, the cross-arm length L_b is fixed at the nominal value listed in Appendix A-C rather than treated as a free model parameter. [**TODO:** Determine and justify the operating range of Y; distinguish the mechanical stroke, simulated-data range, and range covered by the measured ILC records.]

B. Self-scheduled LPV-LFR representation

- Y is a state and the scheduling variable; $M(Y)^{-1}$ makes the model rational in Y (Drenth)
- $M(Y) = M_0 + YM_1 + Y^2M_2$; Y enters only through $Y\ddot{q}$ and $Y^2\ddot{q}$, realized by repeating Y (this work)
- Latent variables z, w and Delta-block $\Delta(Y) = YI_6$; not claimed minimal (this work)
- LTI part G in the continuous-time form of Drenth Eq. (6); its first block row is the state equation (adapted)
- Closing the algebraic loop gives back (2) exactly (this work)
- Well posed iff $M(Y)$ nonsingular, true for every Y; closed-form $M(Y)^{-1}$, no numerical solve (Drenth Def. 1; this work)

The measured payload position Y is both a state and the scheduling variable, making the baseline a self-scheduled LPV model [**TODO:** source, or do i state it earlier..?]. Solving (2) for $\ddot{q}$ requires $M(Y)^{-1}$, so the model is rationally dependent on Y. An Linear Fractional Representation (LFR) is introduced to separate this rational scheduling dependence. We move the dependence out of the LTI part G and into a Delta-block $\Delta(Y)$, with the scheduling being connected in feedback with G (Fig. 2) [8], [16].

To separate the scheduling variable Y, deconstruct the inertia matrix (3) into its Y dependent components,

$$M(Y) = M_0 + YM_1 + Y^2M_2. \quad (5)$$

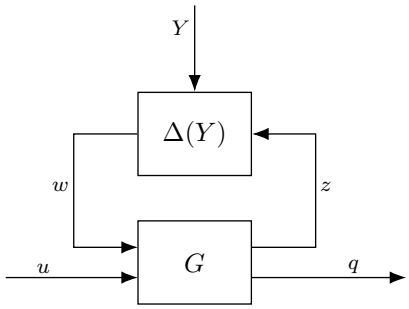


Fig. 2: [**TODO:** Self-scheduled baseline interconnection. The Delta-block closes the latent variables of G through $w = \Delta(Y)z$, with $\Delta(Y) = YI_6$ and Y a component of the state.]

M_0, M_1 and M_2 are given in Appendix A-A. Substituting (5) into (2) gives

$$M_0\ddot{q} + M_1(Y\ddot{q}) + M_2(Y^2\ddot{q}) = u - C\dot{q} - Kq. \quad (6)$$

Here Y only enters through $Y\ddot{q}$ and $Y^2\ddot{q}$, with $Y^2\ddot{q} = Y(Y\ddot{q})$. This lets us apply the same scheduling variable twice instead of introducing a second scheduling variable Y^2,

$$\ddot{q} \xrightarrow{Y} a_1 \xrightarrow{Y} a_2, \quad (7)$$

with $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$.

We treat a_1 and a_2 as independent signals (6) becomes

$$M_0\ddot{q} + M_1a_1 + M_2a_2 = u - C\dot{q} - Kq, \quad (8)$$

which contains no Y. The latent variables are $z = \mathrm{col}(\ddot{q}, a_1)$ and $w = \mathrm{col}(a_1, a_2)$, with the Delta-block mapping z to w.

$$w = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} Y\ddot{q} \\ Ya_1 \end{bmatrix} = \underbrace{\begin{bmatrix} YI_3 & 0 \\ 0 & YI_3 \end{bmatrix}}_{\Delta(Y)=YI_6} \begin{bmatrix} \ddot{q} \\ a_1 \end{bmatrix} = \Delta(Y)z. \quad (9)$$

We do not claim this realization to be minimal [**TODO:** I have an SVD reduction add this in the appendix or not?]. We can write the LTI part G of the LFR in the continous-time form, [16, Eq. (6)],

$$\begin{bmatrix} \dot{\tilde{x}} \\ z \\ q \end{bmatrix} = G \begin{bmatrix} \tilde{x} \\ w \\ u \end{bmatrix}, \quad (10)$$

where $\tilde{x} = \mathrm{col}(q, \dot{q})$ is the state and q the output. The first block row is the state equation, with $\ddot{q}$ obtained from (8) by multiplying with M_0^{-1}. For the blocks of G Appendix A-A. Closing the algebraic loop with (9) results in $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$, which turns (8) back into (2). The LFR is therefore an exact realization of the baseline. The LFR is well posed if the algebraic loop has a unique solution for every Y [16, Def. 1]. For the gantry realization we show that this conditions holds when $M(Y)$ is nonsingular (Appendix A-A). For positive masses and inertias, $M(Y) \succ 0$ for every $Y \in \mathbb{R}$ (Appendix A-D). Therefore the LFR is well posed over the whole Y-operating range. [**TODO:** Connect the two sentences better?]

C. Physical parameters and identifiable combinations

- With q measured, known force scale and full actuation, the input-output behaviour fixes M_0, M_1, M_2, C, K and conversely (this work)
- Twelve nonzero entries, three equal to m_h, leave ten independent functions: exactly ten combinations are structurally identifiable (this work; definition Ovchinnikov)
- Written physically as θ_{base}, the gantry counterpart of base parameters (Gautier and Khalil); one-to-one with the entries for $m_h > 0$ (this work)
- Four lost directions; estimates reported in θ_{base}; result independent of the encoder; augmented model in Section IV; recovery from the records in Setup
- Positive θ_{base} does not give $M(Y) \succ 0$; that holds iff (12) (this work; proof Appendix A-D)
- Training keeps (12) at every iterate; the map and its justification are in Section III-D

The fourteen parameters θ of [**TODO:** needs to have appendix in the name of the table] Table III cannot all be identified from input-output data. The positions X_1, X_2, Y are measured, which gives q through (1), and the forces u are known in newtons. Two parameter sets θ and θ' that produce the same motion for every input force satisfy

$$\big(M(Y) - M'(Y)\big)\ddot{q} + (C - C')\dot{q} + (K - K')q = 0$$

With unconstrained initial conditions and full actuation, any acceleration $\ddot{q}$ can be reached at any state within the operating range. Therefore the identity must hold for all these states and accelerations, this implies that $K = K'$, $C = C'$ and $M(Y) = M'(Y)$. Since $M(Y)$ is quadratic in Y, this also implies that $M_0 = M_0'$, $M_1 = M_1'$ and $M_2 = M_2'$. If these matrices are equal, the input-output behaviour is also the same. Therefore, the part of θ that can be identified is determined by the parameter combinations in M_0, M_1, M_2, C and K. If we ignore the symmetric entries, (3) and (4) contain twelve nonzero entries. Three of these, $M_{0,33}$, $-M_{1,12}$ and $M_{2,22}$, are equal to m_h. This leaves us with ten distinct functions of θ. The Jacobian of these ten functions with respect to $(m_h, d, m_1, m_b, J_b, c_{g1}, c_{g2}, c_{b1}, c_y, k_{b1})$ is block triangular and has a determinant of $m_h L_b^2/2 \neq 0$, meaning that the functions are independent. Since the matrices only depend on these ten functions, exactly ten independent combinations of θ can be identified [**TODO:** check source] [17, Def. 7].
We define these ten combinations as

$$\theta_{\text{base}} = [k_{b,\Sigma}, c_{g1}, c_{g2}, c_y, c_{b,\Sigma}, m_h, m_\Sigma, m_\Delta, J_{\text{eff}}, d]^\top, \quad (11)$$

with $k_{b,\Sigma} = k_{b1} + k_{b2}$, $c_{b,\Sigma} = c_{b1} + c_{b2}$, $m_\Sigma = m_1 + m_2 + m_b$, $m_\Delta = m_1 - m_2$ and $J_{\text{eff}} = J_b + J_h + \frac{L_b^2}{4}(m_1 + m_2)$. This follows the idea of base parameters used in robot dynamics [**TODO:** check source] [18]. For $m_h > 0$, θ_{base} can be uniquely determined from the ten entries (Appendix A-B). We therefore estimate θ_{base} instead of θ, presented in the results [**TODO:** Section **??**].

Positive θ_{base} does not imply $M(Y) \succ 0$. For $m_h, m_\Sigma, J_{\text{eff}} > 0$, $M(Y) \succ 0$ for every Y if and only if

$$m_\Sigma J_{\text{eff}} > \tfrac{L_b^2}{4}\, m_\Delta^2 \quad (12)$$

(Appendix A-D). To ensure that well-posedness and (12) hold during training, we use a logarithmic parameterization for the nine positive combinations and a $\tanh$-bounded m_Δ (Section III-D).

III. Dynamic Augmentation of the LPV-LFR Baseline

- The section extends the baseline with learned dynamics and identifies the result from closed-loop records; it names its five subsections (style profile)
- Contribution share: the realisation on the self-scheduled gantry baseline with joint estimation of θ_{base}, its identification in closed loop with the known controller, and the PS initialisation of the added states (MUST ESTABLISH)

The dynamics of the system that are not captured by the baseline model defined in Section II-B will be identified from input-output data measured in closed-loop. The baseline model will be augmented to represent these missing dynamics. A correction to the states of the baseline model alone is not sufficient, so additional states are introduced. We make use of a neural network to model both the changes for the baseline states and the additional states.
The model is trained using short windows of the closed-loop data, where each window is simulated with the known controller. The initial states are estimated by an encoder, including the additional states. During training, the physical parameters θ_{base} are also estimated, using the combinations defined in Section II-C.

A. Augmented model

- If the omitted effects are dynamic, the six baseline states cannot represent them; refitting θ_{base} or a static augmentation adds no states (EJC Sec. 2)
- Hoekstra's LFR augmentation structure with a fixed interconnection; the self-scheduled LPV-LFR is its baseline block; the learned block is a network, not Drenth's LPV-LFR augmentation (thesis Eq. (5.2)) (?: reason)
- Parallel because the orthogonal-by-construction method of Section IV is defined for the additive structure (Györök 2026); output map not augmented (Kessels Remark 5.3)
- The structure is a discrete-time state update (EJC Sec. 2, Table 1), so f_{base} is one RK4 step with the input held and $M(Y)$ per stage; the correction is added after the step
- Acyclic interconnection plus the baseline result of Section II: well posed for every Y (EJC Thm. 1)
- Normalisation: affine, statistics of the training records, velocities from differenced positions; later displays in normalised symbols
- One tanh MLP supplies f_{aug} and g_{aug} (EJC Sec. 4.3); it corrects every physical row, so a persistent correction on X or Y accumulates in position (?: justification); $n_{\bar{x}} = 0$ is the static parallel augmentation

- The baseline/network split is not unique (2026 Sec. 5.2); interpretable means the estimated θ_{base} keeps its physical meaning (Györök Sec. 3); Section IV makes the estimate unique

To represent additional dynamics such as a flexible mode, refitting θ_{base} or correcting the baseline states with a static augmentation is not sufficient. Both these methods [**TODO:** only retain] the original states of the baseline, while missing dynamics require states the baseline does not include [8, Sec. 2]. A dynamic augmentation therefore adds states $\bar{x}$ to model these missing dynamics [8, Sec. 2]. The augmented model updates these states using the full state and input [8, Sec. 2, Table 1]. The LFR-based augmentation structure of Hoekstra et al. is used, an interconnection matrix connects a baseline block and a learning block [8, Sec. 3.1]. The learning components are a neural network consisting of f_{aug} and g_{aug}. [**TODO:** constant block is not really constant] Similar to the LFR in Section II-B (Fig. 2), a constant matrix connects the two blocks through latent variables. The self-scheduled LFR The baseline block for this is the self-scheduled LFR of Section II-B, this is a nonlinear [**TODO:** Check can i state nonlinear here] map of the state and the input [19, Sec. 2.4]. The augmented model is thus an LFR whose baseline block is itself an LFR in Y.

The dynamic-parallel augmentation structure is chosen, because the orthogonal-by-construction parametrisation that Section IV builds upon is defined for the additive structure [11].

The output map is not augmented (no h_{aug}), because (1) gives the exact relation between the states and the measured outputs. This is also done to ensure that q still represents the physical positions. With a learned correction to the output map the states could stop representing the actual positions. Kessels also leaves the output map unchanged when it is already accurate [13, Remark 5.3]. [**TODO:** not an LPV-LFR with a learned scheduling map as in the augmentation of Drenth [19, Eq. (5.2)].] [**TODO:** Missing: reason for a network as learning block rather than Drenth's LPV-LFR augmentation. The research plan (Aspect 2) extends the structure "to the LPV setting following" Drenth and leaves "the specific parameterization to be established". Candidates: (a) Hoekstra's framework with its zero start and well-posedness result is kept as is; (b) the research question asks only for a structure that captures the omitted dynamics. Close D-017 with a decision entry.]

The model is defined in discrite time with the augmentation added to the state update of the baseline [8, Sec. 2, Table 1]. This gives us

$$\begin{aligned}\tilde{x}_{k+1} &= f_{\text{base}}(\theta_{\text{base}}, \tilde{x}_k, u_{\text{act},k}) + f_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}),\\ \bar{x}_{k+1} &= g_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}),\\ \hat{y}_k &= P^\top q_k,\end{aligned} \tag{13}$$

where $\tilde{x}_k = \text{col}(q_k, \dot{q}_k) \in \mathbb{R}^6$ is the physical state of (10), $\bar{x}_k \in \mathbb{R}^{n_{\bar{x}}}$ the added state, $\hat{x}_k = \text{col}(\tilde{x}_k, \bar{x}_k)$, $u_{\text{act},k} \in \mathbb{R}^3$ the actuator forces, $\hat{y}_k \in \mathbb{R}^3$ the predicted actuator positions, and $y_k = q_{\text{act},k}$ the measured positions, $\theta_{\text{base}} \in \mathbb{R}^{10}$ the combinations (11), and [**TODO:** using theta might be confusing with baseline] $\theta_{\text{aug}} \in \mathbb{R}^{n_{\text{aug}}}$ the network parameters. The added states are updated by g_{aug} without any predetermined physical meaning, in contrast to the baseline states $\tilde{x}$ for which the positions are measured.

The baseline is evaluated over one sample period T_s. Writing (2) in state-space form gives

$$\dot{\tilde{x}} = F(\tilde{x}, u) = \begin{bmatrix} \dot{q} \\ M(Y)^{-1}\big(u - C\dot{q} - Kq\big) \end{bmatrix}, \tag{14}$$

where $u = Pu_{\text{act}}$ is the generalized force (1), M, C and K depend on θ_{base} through (11), and $M(Y)^{-1}$ is the closed loop of the LFR of Section II-B. f_{base} is calculated using one fourth-order Runge-Kutta (RK4) step of F over T_s, with the input kept constant during the step. For each of the four RK4 steps $M(Y)$ is evaluated using the value of Y from its intermediate state.

[**TODO:** The augmented model admits a unique solution for every state, input and Y. The interconnection has no algebraic loops, making it acyclic, and both blocks are twice continuously differentiable, which together give well-posedness [8, Cond. 6, Cond. 8, Thm. 9]. The baseline LFR is also well posed (Section II-B).]

The inputs to the neural network are normalised to zero mean and unit variance, which has been shown to improve the estimation of models containing neural networks [8, Sec. 5.3]. The mean and standard deviations are computed from the training data. For the baseline states the measured positions are converted to their generalized positions, for which the velocities can be estimated from their finite difference by dividing with T_s. This ensures that the positions and outputs are scaled with the same measurements: keeping the scales consistent. This also makes sure that all models independent of its initial parameters use the same scale,

$$\tilde{x}^{\text{n}} = T_x(\tilde{x} - \mu_x), \quad u^{\text{n}} = T_u(u_{\text{act}} - \mu_u), \quad y^{\text{n}} = T_y(y - \mu_y), \tag{15}$$

where $\mu_x \in \mathbb{R}^6$ and $\mu_u, \mu_y \in \mathbb{R}^3$ are the channel means, and $T_i = \text{diag}(\sigma_i)^{-1}$ for $i \in x, u, y$, with σ_i the corresponding standard deviations. The additional states $\bar{x}$ are not normalised, so $\hat{x}^{\text{n}} = \text{col}(\tilde{x}^{\text{n}}, \bar{x})$.

After normalisation the model equations become

$$\begin{aligned} f^{\text{n}}_{\text{base}}(\theta_{\text{base}}, \tilde{x}^{\text{n}}, u^{\text{n}}) &= T_x\big(f_{\text{base}}(\theta_{\text{base}}, \tilde{x}, u_{\text{act}}) - \mu_x\big),\\ f^{\text{n}}_{\text{aug}} &= T_x f_{\text{aug}}, \qquad g^{\text{n}}_{\text{aug}} = g_{\text{aug}},\\ \hat{y}^{\text{n}}_k &= C^{\text{n}}\tilde{x}^{\text{n}}_k, \qquad C^{\text{n}} = T_y P^\top [I_3 \;\; 0]\, T_x^{-1}. \end{aligned} \tag{16}$$

where $\tilde{x}$ and u_{act} follow from $\tilde{x}^{\text{n}}$ and u^{n} by inverting (15), so RK4 uses the states and inputs in physical units. The correction represents a change in the states itself and not the full state itself, and therefore we only scale it. The output map remains linear because the mean of the positions is calculated from the output mean as $P^{-\top}\mu_y$.

A multilayer perceptron ϕ_{aug} with tanh activation functions for the hidden layers and a linear output layer is used for both $f^{\text{n}}_{\text{aug}}$ and $g^{\text{n}}_{\text{aug}}$:

$$\text{col}(f^{\text{n}}_{\text{aug}}, g^{\text{n}}_{\text{aug}}) = \phi_{\text{aug}}(\theta_{\text{aug}}, \hat{x}^{\text{n}}_k, u^{\text{n}}_k).$$

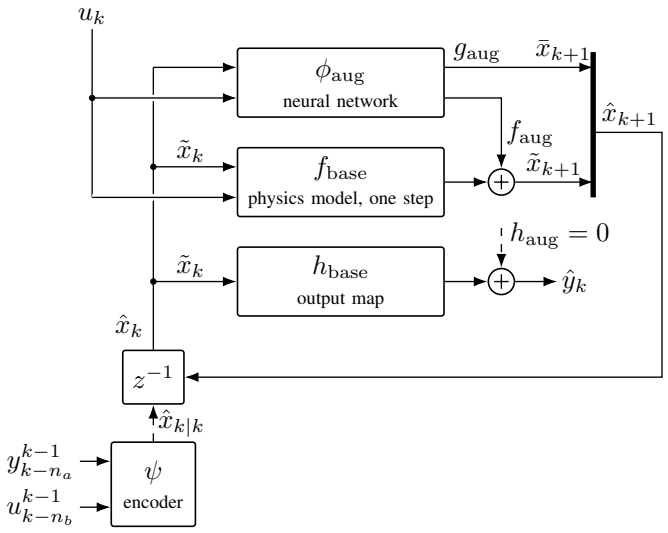


Fig. 3: Augmented model. The physics step f_{base} and the network ϕ_{aug} act in parallel on $\hat{x}_k$. ϕ_{aug} gives f_{aug} and g_{aug}, and the output map is not learned. The encoder ψ sets $\hat{x}$ at each window start. During training, the input is the closed-loop input of Section III-C. [**TODO:** No setpoint generator currently for the inputs; layout cramped at one column width.]

The inputs to the neural network are the $6 + n_{\bar{x}}$ states and the three actuator forces. The first six outputs $f_{\text{aug}}^{\text{n}}$ update the six physical states and the remaining $n_{\bar{x}}$ outputs $g_{\text{aug}}^{\text{n}}$ and equals the next additional states $\bar{x}_{k+1}$ (Fig. 3). These corrections act on all six physical states, the positions included, as in the dynamic-parallel structure [8, Table 1]. The positions are also corrected because within one sample period an unmodelled force would change both the velocity and position, correcting only for the velocities would not be able to capture this. Both f_{aug} and g_{aug} are given the payload position Y through $\tilde{x}_k$, enabling the neural network to learn model position-dependent effects.

Randomly initializing the learning components can give an unstable and bad optimization problem, to resolve this training starts from the baseline [8, Sec. 5.4]. At the start of training f_{aug} is set to zero, resulting in the augmented model being equal to the baseline. Because we make use of a feedforward neural network this only requires us to zero the output layer.

The gantry baseline model has two free axes, X and Y, without stiffness in (4), which are also affected by the learned correction. The discretized baseline model has a pole at $z = 1$ on each of these stiffness free axes, making it marginally stable. Therefore these axes act as integrators: a consant error in the correction keeps accumalating in the position. The alternative to only correct the velocities would not prevent the integrator issue, because an error in the velocity also accumulates in the position. The feedback controller is able to correct this drift, which is why we have chosen to simulate the model in closed-loop (Section III-C).

B. Encoder and initial state

- Only the positions are measured, so an encoder estimates all $6 + n_{\bar{x}}$ states at each window start from the input-output window (Beintema Sec. 3.1); the window ends at $\tau - 1$ (2026 Eq. (22e), D-250)
- The encoder is a linear map plus a network correction, as encoder paper Eq. (8), with no separate bias (the output bias of ψ carries any constant)
- The physical rows start data-based: the baseline is self-scheduled by the state Y, hence nonlinear; a reconstruction from a linearisation is accurate only near its linearisation point (encoder paper Sec. 3.1), and the paper suggests the data-based route for nonlinear baselines (Secs. 3.2, 4.4)
- The physical rows are fitted to a baseline simulation as in 2026 Eq. (30); the simulation runs in closed loop because the open-loop baseline drifts on X and Y (this work, D-251)
- The fitted physical rows only set the start and are trained jointly with θ_{base} (2026 Eq. (22)); current runs: model-based map at $Y = 0$ (revnote, appendix)

- Only positions are measured, so an encoder estimates the state at each window start τ from past inputs and outputs
- Choice: short windows from encoded states
- The history ends at sample $\tau - 1$, as in Hoekstra et al.; the encoder estimates all $6 + n_{\bar{x}}$ states:

Each simulation window starts from an initial state $\hat{x}_\tau$, but we measure only the positions of the physical state $\tilde{x}$. The additional states $\bar{x}$ are latent. Estimating $\hat{x}_\tau$ as a separate parameter for each window would increase the number of parameters as the data grows [20, Sec. 3.1]. Following Beintema et al., we use an encoder that estimates $\hat{x}_\tau$ from past inputs and outputs [20, Sec. 3.2]. Simulating the short windows using the encoded states keeps the computation per gradient step independent of the record length, and stabilises optimization [20, Sec. 3.1]. The encoder uses the samples up to $\tau - 1$, as in [8, Eq. (22e)]:

$$\begin{aligned} \hat{x}_{\tau|\tau}^{\text{n}} &= \psi(\theta_{\text{enc}}, \chi_\tau), \\ \chi_\tau &= \text{col}\big(y_{\tau-1}^{\text{n}}, \ldots, y_{\tau-n-1}^{\text{n}}, u_{\tau-1}^{\text{n}}, \ldots, u_{\tau-n-1}^{\text{n}}\big), \end{aligned} \tag{17}$$

where $n + 1$ is the number of samples in the window, $\chi_\tau \in \mathbb{R}^{6(n+1)}$ stacks the normalised recorded outputs and inputs, and $\theta_{\text{enc}} \in \mathbb{R}^{n_{\text{enc}}}$ is the parameter vector of the encoder. The encoder estimates all $6 + n_{\bar{x}}$ initial states $\hat{x}_\tau$, which includes the measured positions. We estimate the positions instead of using the measured y_τ to keep the positions consistent with the estimated velocities.

[**Note:** The current results use the window up to and including τ (D-054), so the first term of (25) scores a sample the encoder has seen. The final runs will use the window up to $\tau - 1$ as described here.]

- The encoder is a linear map plus a network correction; W^b gives the physical states, W^a the additional states:

The encoder uses a linear map with a nonlinear correction [21, Eq. (8)]:

$$\psi(\theta_{\text{enc}}, \chi_\tau) = \begin{bmatrix} W^b \\ W^a \end{bmatrix} \chi_\tau + \tilde{\psi}(\chi_\tau), \tag{18}$$

where $W^b \in \mathbb{R}^{6\times 6(n+1)}$ and $W^a \in \mathbb{R}^{n_{\bar{x}}\times 6(n+1)}$ are the linear maps for the six physical states and the $n_{\bar{x}}$ additional states, and $\tilde{\psi}$ is a neural network. The encoder parameters θ_{enc} are W^b, W^a and the parameters of $\tilde{\psi}$. The first six outputs ψ_b estimate the physical states and the remaining $n_{\bar{x}}$ outputs ψ_a estimate the additional states. We use θ^a_{enc} for the parameters that affect only ψ_a: W^a and the additional-state rows of the output layer of $\tilde{\psi}$. [**Note:** The current results apply W^b and W^a to the uncentred window $\chi_\tau + \chi_0$, where χ_0 restores the means that (15) removes, and subtract $T_x\mu_x$ from the physical rows. This amounts to a fixed bias $W^b\chi_0 - T_x\mu_x$ on the physical rows and a constant $W^a\chi_0$ on the initial additional states. The final runs will use the centred window without a separate bias, as described here; the fitted output bias of $\tilde{\psi}$ carries any constant.] [**TODO:** VERIFY bib: `hoekstra2026encoder` = arXiv:2602.13108v1 (13 Feb 2026), Hoekstra, Györök, Tóth, Schoukens; matches PDF p. 1. Check for a published (IFAC) version before submission.]

- Choice: the physical rows start data-based, because the baseline is nonlinear (LPV in Y):

The physical rows are initialized from the baseline model. Because solving (2) for $\ddot{q}$ requires $M(Y)^{-1}$ and Y is a state (Section II-B), the baseline is nonlinear. To create a model-based state reconstruction map for the nonlinear baseline, a linearisation point has to be chosen, making the map only accurate around that point [21, Sec. 3.1]. Hoekstra et al. therefore suggest fitting the encoder to states simulated with the baseline [21, Sec. 4.4, Eq. (35)], as their augmentation method does [8, Eq. (30)]. We apply this approach and simulate the baseline model once before training to obtain the physical state at every sample τ. The recorded actuator forces u_{act} and positions y from the training records with the known controller in (23) are used to simulate the physical states in closed-loop. Because in open-loop the baseline drifts on the free axes X and Y (Section III-A):

$$\tilde{x}^{\text{n},0}_{k+1} = f^{\text{n}}_{\text{base}}\big(\theta^0_{\text{base}}, \tilde{x}^{\text{n},0}_k, \hat{u}^{\text{n},0}_k\big), \qquad \hat{y}^{\text{n},0}_k = C^{\text{n}}\tilde{x}^{\text{n},0}_k, \tag{19}$$

where $\tilde{x}^{\text{n},0}_k$ is the simulated baseline state, θ^0_{base} are the initial parameters, and $\hat{u}^{\text{n},0}_k$ is the closed-loop input from (23). We use $\hat{y}^{\text{n},0}_k$ as the model output and set the controller state x^c to zero at the start of each record (see Section III-C). [**TODO:** Missing: initial state of (19) per record (D-251 open). Candidate: the measured positions with zero velocity at the record start (basis: the records start at rest, to be checked in the data).]

- Choice: the physical rows of the encoder are fitted to the simulated states, as in Hoekstra et al.:

The physical rows of the encoder are then fitted to the simulated states [8, Eq. (30)]:

$$\theta^{b,0}_{\text{enc}} \in \arg\min_{\theta^b_{\text{enc}}} \frac{1}{|\mathcal{D}|}\sum_{\tau\in\mathcal{D}} \big\|\psi_b(\theta^b_{\text{enc}}, \chi^0_\tau) - \tilde{x}^{\text{n},0}_\tau\big\|_2^2, \tag{20}$$

where θ^b_{enc} consists of W^b, the hidden-layer parameters of $\tilde{\psi}$ and the physical rows of its output layer. These parameters are fitted starting form the weights drawn from $\mathcal{U}(-1/\sqrt{m}, 1/\sqrt{m})$, where m is the input size of the layer [21, Eq. (35)]. The fitted values $\theta^{b,0}_{\text{enc}}$ are used to start training. χ^0_τ is formed as we do in (17) using $\hat{y}^{\text{n},0}$ and $\hat{u}^{\text{n},0}$. $\mathcal{D}$ is the set of samples τ in the training records for which a complete window is available. We use the closed-loop input from (19), instead of the recorded input as in [8, Eq. (30)].

- $\theta^{b,0}_{\text{enc}}$ only sets the start; θ^b_{enc} is trained jointly with θ_{base}

We only use $\theta^{b,0}_{\text{enc}}$ for initialization. θ^b_{enc} is trained in combination with θ^a_{enc}, the network parameters θ_{aug} and the baseline parameters θ_{base}, which the encoder paper keeps fixed [21, footnote 1]. [**TODO:** Fix before the final run (`FINAL-RUN-FIXES.md` item 1): W^b is built from the nominal parameters, which equal those of the simulated truth, while θ_{base} starts detuned. The final runs build W^b at the start θ^0_{base}; the text then states that.] [**Note:** The current results build W^b from the true parameters of the simulated system, while θ_{base} starts detuned. The final runs will simulate the baseline at the start θ^0_{base} as described here.] [**Note:** The current results initialise only W^b, with the model-based reconstructability map [21, Eqs. (15) to (17)] of the baseline linearised at rest at $Y = 0$, and start the whole output layer of $\tilde{\psi}$ at zero. The final runs will fit the physical rows as described here (D-251).]

C. Closed-loop simulation

- The model is not driven by the recorded input (2026 Eq. (22)), for two reasons: noise-induced bias under feedback (Kessels Remark 5.6, SUBNET; indirect approach, Forssell and Ljung) and the free axes of Section III-A, which violate the stability condition behind SUBNET consistency (Beintema Cond. 1); motivation, not a proof
- As in Kessels (Eqs. (5.12), (5.13)), each window is simulated in closed loop with the known controller, which forms its error against the model output; system and model loops share r and u^{ff}
- The controller is linear and time invariant, so subtracting the system's loop cancels r and u^{ff}; the residual form needs only the recorded u_{act} and y, and is exact when u_{act} was produced by this controller from y (this work)
- $x^c_\tau = 0$ starts the model's controller in the system's controller state, as the encoder does for the plant state; no controller state is estimated, unlike Kessels Remark 5.4 (this work)
- No plant feedthrough fixes the step order, so D_{fb} closes no algebraic loop (as Kessels Remark 5.1); one normalised step displayed
- The output error re-enters the model input, so part of it is corrected within the window; error before τ is not carried in, the free-run validation carries it

The data is measured in closed-loop, and the model is simulated in closed-loop. Instead of using the recorded input as in [8, Eq. (22)], we include the feedback controller in the simulation for the following reasons. The feedback controller

causes a correlation between the recorded input and the measurement noise. Kessels was not able to identify an open-loop SUBNET model using closed-loop data from a wire bonder. Kessels cites earlier work suggesting that this could be explained by noise-induced bias due to this correlation and the integral term of the controller [13, Remark 5.6].

The indirect approach for linear systems with a known controller fits the closed-loop model using the reference. Because the reference is uncorrelated with the noise, the model can be identified from the output error [22, Sec. 5.2.1]. [**TODO:** output prediction error??]

Also, as has been established in Section III-A: the free axes X and Y make the open-loop gantry marginally stable. SUBNET assumes that the system that generates the data is incrementally exponentially output stable [20, Cond. 1]. The identification method we have adapted from [8] relies on this SUBNET assumption [8, Sec. 5.5]. The open-loop gantry system does not satisfy this assumption: a mismatch in the initial velocity will create a permanent position offset. In closed-loop the feedback controller is able to correct this offset. These reasons motivate the choice for simulating in closed-loop. They do not prove that our estimate is consistent. [**TODO:** ?] [**TODO:** Missing: D-140 (Ruled out (4)) bars Kessels' Remark 5.6 as an argument because the records were then noise free. The thesis records are noisy, so the ban appears lapsed. Candidate: keep the sentence and amend D-140 (basis: noisy thesis records, Section V).] [**TODO:** Missing: evidence on the thesis data that open-loop rollouts accumulate position error on X and Y. Candidate: the open-loop versus closed-loop drift on one record that `THESIS-RESULTS.md` Sec. 5 plans, reported in Section VI.]

Every training window is simulated in closed-loop with the known controller, as in Kessels [13, Eqs. (5.12), (5.13)]. The controller is linear and time invariant, the model output is used to calculate the position error in actuator coordinates [**TODO:** qact?]. The controller calculates the actuator forces from this error. The system and model use the same reference r and feedforward u^{ff}, but use their own output for feedback, with controller states x^{fb} and $\hat{x}^{\mathrm{fb}}$ (Fig. 4):

$$\begin{aligned} x^{\mathrm{fb}}_{k+1} &= A_{\mathrm{fb}} x^{\mathrm{fb}}_k + B_{\mathrm{fb}}(r_k - y_k), \\ u_{\mathrm{act},k} &= C_{\mathrm{fb}} x^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - y_k) + u^{\mathrm{ff}}_k, \\ \hat{x}^{\mathrm{fb}}_{k+1} &= A_{\mathrm{fb}} \hat{x}^{\mathrm{fb}}_k + B_{\mathrm{fb}}(r_k - \hat{y}_k), \\ \hat{u}_{\mathrm{act},k} &= C_{\mathrm{fb}} \hat{x}^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - \hat{y}_k) + u^{\mathrm{ff}}_k, \end{aligned} \tag{21}$$

where $(A_{\mathrm{fb}}, B_{\mathrm{fb}}, C_{\mathrm{fb}}, D_{\mathrm{fb}})$ are the state-space matrices of the controller discretised with sampling time T_s. We assume that the system uses the same controller. Both r_k and u^{ff}_k are in actuator coordinates. The model uses the calculated actuator force $\hat{u}_{\mathrm{act},k}$ instead of the recorded $u_{\mathrm{act},k}$ in (13).

Since we use a linear controller we are able to subtract the system and model controller equations to cancel r and u^{ff}. This leaves us with the controller state difference $x^c = \hat{x}^{\mathrm{fb}} - x^{\mathrm{fb}}$, driven by $y - \hat{y}$. The model is still simulated with respect to the reference r, but r and u^{ff} are not used in simulation. If the controllers between the system and simulation differ, the equations are only approximate and noise-induced bias cannot be ruled out.

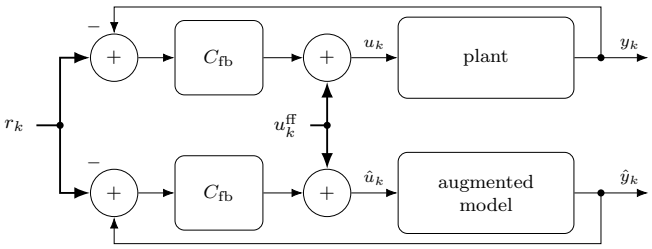


Fig. 4: Recorded system loop (top) and augmented model loop (bottom), both using the same feedback controller, reference r_k and feedforward u^{ff}_k. The model output is calculated before the input updates the model state.

At the start of each training window we set $x^c_\tau = 0$, which sets the model's controller state equal to that of the system at τ. Therefore the encoder only has to initialize the states of the model. Kessels instead needs the system's controller state at the start of each window, either measured directly or calculated by simulating the controller from the start of the record [13, Remark 5.4]. We then write the closed-loop model in (21) as

$$\begin{aligned} x^c_{k+1} &= A_{\mathrm{fb}} x^c_k + B_{\mathrm{fb}}(y_k - \hat{y}_k), \qquad x^c_\tau = 0, \\ \hat{u}_{\mathrm{act},k} &= u_{\mathrm{act},k} + C_{\mathrm{fb}} x^c_k + D_{\mathrm{fb}}(y_k - \hat{y}_k), \end{aligned} \tag{22}$$

which only needs the recorded u_{act} and y. [**TODO:** Should be discussed in the setup for controller discretization] [**TODO:** Check for Setup: the training loop uses the controller re-discretised at the model rate, while the records were generated at a higher rate. Run the re-discretised controller on the recorded $r - y$ and compare its feedback force with the recorded one. A rollout with $\hat{y} = y$ gives $\hat{u} = u$ by construction and checks nothing. Related: `FINAL-RUN-FIXES.md` item 3, the D-232 leak check on the lowpass data, passed (0.22 to 0.50 nm).]

The model output depends only on the state. For each sample we first calculate the output and then the error and model input before updating the states. This avoids an algebraic loop through D_{fb}, for the same reason as Kessels [13, Remark 5.1]. For a window starting at τ, the model equations in the normalised coordinates of (15) are

$$\begin{aligned} \hat{y}^{\mathrm{n}}_{k|\tau} &= C^{\mathrm{n}} \tilde{x}^{\mathrm{n}}_{k|\tau}, \qquad \varepsilon_{k|\tau} = T_y^{-1}\big(y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau}\big), \\ \hat{u}^{\mathrm{n}}_{k|\tau} &= u^{\mathrm{n}}_k + T_u\big(C_{\mathrm{fb}} x^c_k + D_{\mathrm{fb}} \varepsilon_{k|\tau}\big), \\ x^c_{k+1} &= A_{\mathrm{fb}} x^c_k + B_{\mathrm{fb}} \varepsilon_{k|\tau}, \qquad x^c_\tau = 0, \end{aligned} \tag{23}$$

and the next model state $\hat{x}^{\mathrm{n}}_{k+1|\tau}$ is calculated using (16) with input $\hat{u}^{\mathrm{n}}_{k|\tau}$. Each window starts from the encoder state in (17). The output error $\varepsilon_{k|\tau} = y_k - \hat{y}_{k|\tau}$ is in metres.

The controller uses the output error to correct the model input within the training window, as in the closed loop system. Each window starts from a new encoder estimate and $x^c_\tau = 0$, so the window is not affected by model errors before τ. The validation of Section III-D is simulated in closed-loop and only initialized at the start.

D. Identification

- θ_{base} is estimated jointly with network and encoder, as in 2026 Eq. (22); Kessels trains only the network weights (Eq. (5.12))
- Training coordinates β, zero at θ^0_{base}: nine combinations logarithmic, m_Δ through (24); every β admissible, so no clamp or penalty (this work, D-229; adapted from Drenth Sec. 4.2)
- Criterion (25): Eq. (22) adapted by the loop of Section III-C and the encoder of Section III-B; every window sample scored; controller fixed; a closed-loop fit can mask plant error (Section VI)
- The baseline/network split is not unique (2026 Sec. 5.2), so θ_{base} is not interpretable in the sense of Section I; Section IV addresses this (unique to first order only?)
- Adam, then an unbounded L-BFGS polish (adapted from Drenth Sec. 5.1); checkpoint with the lowest closed-loop free-run error over whole validation records; polish kept only if it lowers that error
- Initialisation (26): $\tilde{x}$ and $\hat{y}$ reproduce the baseline, $\bar{x}$ evolves through a live g_{aug} (2026 Eq. (29)); adapted from Sec. 5.4.3 (random added-state rows, no linear bypass); the model is named in Section V, not here

The network, encoder and θ_{base} are jointly estimated, as in [8, Eq. (22)]. The optimiser updates θ_{base} using $\beta \in \mathbb{R}^{10}$. The nine positive parameter combinations are $\theta_{\text{base},j} = \theta^0_{\text{base},j}\, e^{\beta_j}$, while m_Δ can be positive or negative and is bounded by (12):

$$m_\Delta = \rho \frac{2}{L_b} \sqrt{m_\Sigma J_{\text{eff}}}\, \tanh\big(\eta_0 + s\,\beta_\Delta\big), \tag{24}$$

where β_Δ determines m_Δ and $\rho \in (0,1)$ provides a safety margin. We choose the offset η_0 and scale s so that $\beta = 0$ gives $m_\Delta = m_{\Delta,0}$ and $\partial m_\Delta / \partial \beta_\Delta = m_{\Delta,0}$. This matches the logarithmic parameters, where $\partial \theta_{\text{base},j} / \partial \beta_j = \theta^0_{\text{base},j}$ at $\beta = 0$ (Appendix A-D). The initial value must satisfy $|m_{\Delta,0}| < \rho(2/L_b)\sqrt{m_{\Sigma,0} J_{\text{eff},0}}$. The bound in (12) holds for every β, therefore no clamping or penalty is needed.

The parameters $\vartheta = (\beta, \theta_{\text{aug}}, \theta_{\text{enc}})$ are trained to minimise the normalised output error of the closed-loop simulation, with the controller fixed. We adapt this from [8, Eq. (22)], adapted for the closed loop of Section III-C and the encoder of Section III-B:

$$\begin{aligned}
&\min_{\vartheta}\ V(\vartheta) = \frac{1}{|\mathcal{W}|\, n_f\, n_y} \sum_{\tau \in \mathcal{W}} \sum_{k=\tau}^{\tau+n_f-1} \left\| y^{\text{n}}_k - \hat{y}^{\text{n}}_{k|\tau} \right\|_2^2 \quad \text{s.t.} \\
&\tilde{x}^{\text{n}}_{k+1|\tau} = f^{\text{n}}_{\text{base}}\big(\theta_{\text{base}}(\beta), \tilde{x}^{\text{n}}_{k|\tau}, \hat{u}^{\text{n}}_{k|\tau}\big) + f^{\text{n}}_{\text{aug}}\big(\theta_{\text{aug}}, \hat{x}^{\text{n}}_{k|\tau}, \hat{u}^{\text{n}}_{k|\tau}\big), \\
&\bar{x}_{k+1|\tau} = g^{\text{n}}_{\text{aug}}\big(\theta_{\text{aug}}, \hat{x}^{\text{n}}_{k|\tau}, \hat{u}^{\text{n}}_{k|\tau}\big), \\
&\hat{x}^{\text{n}}_{\tau|\tau} = \psi(\theta_{\text{enc}}, \chi_\tau), \quad \hat{y}^{\text{n}}_{k|\tau},\ \hat{u}^{\text{n}}_{k|\tau} \text{ from (23)},
\end{aligned} \tag{25}$$

for $\tau \le k < \tau + n_f$, where each training window starts at a sample $\tau \in \mathcal{W}$ and has length n_f, $n_y = 3$, and $\theta_{\text{base}}(\beta)$ gives the baseline parameters. [**TODO:** might have to move to discussion if i dont add it. Since the controller corrects part of the output error, it can hide errors in the plant model. We therefore also test the model with a different controller in Section VI.]

The baseline parameters and the neural network can both correct the same model errors, making the optimal parameter values not unique [8, Sec. 5.2]. Without enforcing uniqueness the estimated θ_{base} is not interpretable as defined in Section I. Section IV extends the identification to separate what the neural network can learn from the baseline parameters. [**TODO:** Missing: Section IV makes the network orthogonal to the linearised baseline sensitivity at fixed tuples, so the split is unique only to first order around β_{lin}, not a unique minimiser of (25). Candidate: once Section IV states that local result, write "so that the split is unique to first order" with a pointer to it (basis: `04_obc.tex` (32); math check 2026-10-13, F1).]

For training we minimise V with Adam on minibatches of windows, and the gradient is calculated over the full window. We then polish with L-BFGS on a fixed set of windows. This is adapted from the Adam and L-BFGS-B sequence of Drenth et al. [16, Sec. 5.1]. Because every β satisfies (12), unconstrained L-BFGS works without a bound. During the optimization with Adam the parameters with the lowest validation error are kept. The error is the RMS output error in meters for each complete validation record, simulated in closed loop with the initial state estimated once by the encoder and $x^c = 0$. We only keep the polish if it lowers the validation error. [**TODO:** Current runs (`FINAL-RUN-FIXES.md` item 2): the 2026-10-11 runs also reject the polish if `combo_err`, the error of the ten combinations against the true parameters, rises by more than 0.1 %. This uses the simulated truth; the final runs accept on the validation error alone. `param_loss` is zero in all thesis runs (`param_prior` off).]

We start training with the augmented model giving the same output as the baseline model from the same initial state, as required by [8, Eq. (29)]. [**Note:** I use the encoder for my augmented model, so i state: model from the same initial state. But i could leave it out.] Therefore we zero the physical rows of ϕ_{aug} its output layer, and the added-state rows are kept random:

$$\begin{aligned}
&\text{baseline:} && \beta = 0, \\
&\text{network:} && W_L^{\tilde{x}} = 0, \quad b_L = 0, \quad W_L^{\bar{x}} \sim \mathcal{U}(-c, c), \\
&\text{encoder:} && \theta^{b,0}_{\text{enc}} \text{ from (20)}, \\
& && W^a \sim \mathcal{U}(-a, a), \quad \tilde{W}_L^{\bar{x}} = 0, \quad \tilde{b}_L^{\bar{x}} = 0, \\
& && c = 1/\sqrt{n_h}, \quad a = \sqrt{6/(n_{\bar{x}} + 6(n+1))},
\end{aligned} \tag{26}$$

where W_L and b_L are the output layer weights and bias of ϕ_{aug}, with rows $W_L^{\tilde{x}}$ for $f^{\text{n}}_{\text{aug}}$ and $W_L^{\bar{x}}$ for $g^{\text{n}}_{\text{aug}}$. The number of neurons per hidden layer n_h, $\tilde{W}_L^{\bar{x}}$ and $\tilde{b}_L^{\bar{x}}$ are the added-state rows of the output layer weights and bias of $\tilde{\psi}$. We initialise $W_L^{\bar{x}}$ and the hidden layers of ϕ_{aug} from $\mathcal{U}(-1/\sqrt{m}, 1/\sqrt{m})$, where m is the input size of the layer. We use Xavier initialisation for W^a, following [8, Eq. (31)]. We choose the uniform distribution with gain one, giving $a = \sqrt{6/(n_{\bar{x}} + 6(n+1))}$. At the start of training, $\theta_{\text{base}} = \theta^0_{\text{base}}$ and $f^{\text{n}}_{\text{aug}} = 0$. The augmented model and baseline model give the same $\tilde{x}$ and $\hat{y}$, given they start from the same physical state. The added states $\bar{x}$ are still updated by $g^{\text{n}}_{\text{aug}}$, as in [8, Eq. (29)].

[**Note:** The current results start W^b from the model-based reconstructability map [21, Eqs. (15) to (17)] of the baseline linearised at rest at $Y = 0$, and the whole output layer of $\tilde{\psi}$ at zero. The final runs will start the physical encoder rows from $\theta^{b,0}_{\text{enc}}$ of (20) as described here.] [**Note:** The current results start with the whole output layer of ϕ_{aug} at zero, so $g_{\text{aug}} = 0$ and the added states are zero after the first step. The final runs will use live g_{aug} as described here.] [**TODO:** Current runs: the 2026-10-11 runs (R1 to R7) set the whole output layer of ϕ_{aug} to zero (`G_INIT` zero, D-240), as the authors' public code does (D-237), so $g^{\text{n}}_{\text{aug}} = 0$ and $\bar{x}$ is zero after the first step. The final runs use (26) (D-248, `FINAL-RUN-FIXES.md` item 6).]

E. Added-state initialisation

- Problem: the start (26) gives $\tilde{x}$ a model (f_{base}, W^b, output) and $\bar{x}$ none (random W^a, random added-state rows of ϕ_{aug}); V scores only the output and f_{aug} can lower the same error statically, so training can leave the omitted dynamics out of $\bar{x}$ (Venkatraman Sec. 2.1; Section VI tests it)
- PS is this work's, adapted from two-stage regression (Hefny Sec. 2) and its use before BPTT (Downey Sec. 4.2); it gives $\bar{x}$ a start from data, as W^b gives $\tilde{x}$ one from the baseline
- S1: a decoder on ψ_a predicts the current model's residual over $\tau + 1$ to $\tau + H$; the minimiser is the conditional mean, so $\bar{x}_{\tau|\tau}$ becomes a predictive state; trains θ_{p}, θ^a_{enc} only, one update after every update of V
- Stopping rule: ν on calibration windows ends S1 (plateau or cap) and decides whether S2 runs
- S2: once, one-step regression of $g^{\text{n}}_{\text{aug}}$ on frozen encoder pairs with the recorded input, which equals the loop input when the encoder reproduces y_{k_i}; overlapping windows (?)
- Hand-over: decoder discarded, (25) continues from ϑ_{PS}; deployed model, criterion and selection unchanged; $\bar{x}$ latent; $\mathcal{M}_{\text{PS}}$ named
- The current runs differ in two points: g_{aug} starts at zero, and the S1 target starts at τ (?)

The physical states of the baseline are initialized with the data-based fit (20) and updated through $f^{\text{n}}_{\text{base}}$, while the added-state rows W^a are randomly initialized. For the physical states their generalized coordinates q_k are directly mapped to actuator positions, $\hat{y}_k = P^\top q_k$ in (13). The added states are updated through the random added-state rows of ϕ_{aug}, without their initialization or initial updates being derived from the baseline or the data. The output $\hat{y}_k = P^\top q_k$ contains no added state, and $\bar{x}_k$ enters $\bar{x}_{k+1}$ only through $f^{\text{n}}_{\text{aug}}$ in (25). This means that for the added states V only updates their effect on the output through $f^{\text{n}}_{\text{aug}}$. From (25), V contains nothing that directly incentivises $\bar{x}$ to predict the resulting output error. This gives $f^{\text{n}}_{\text{aug}}$ the capability to lower part of this error with $\tilde{x}$ and $\hat{u}$ alone. Minimising V can therefore reduce the error without capturing the missing dynamics in $\bar{x}$. [**TODO:** Section VI tests whether it does.] This forms a sharp contrast between the physical and added states, due to the physical states being

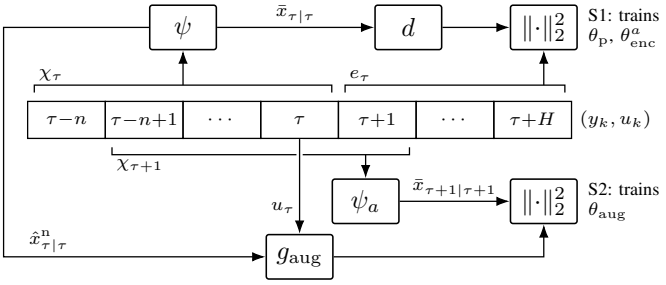


Fig. 5: Windows of PS on one record, layout adapted from [23, Fig. 2]. S1 (top): the encoder maps the window χ_τ to $\hat{x}^{\text{n}}_{\tau|\tau}$, and the decoder d maps its added part $\bar{x}_{\tau|\tau}$ to the residuals e_τ of the next H samples (27). S2 (bottom): g_{aug} maps $\hat{x}^{\text{n}}_{\tau|\tau}$ and the recorded u_τ to the added state encoded from the window shifted by one sample (29). [**TODO:** The figure shows the target from $\tau + 1$ (`FINAL-RUN-FIXES.md` item 5). The 2026-10-11 runs start it at τ, so the noise in y_τ enters both χ_τ and e_τ.]

able to directly map to the output, while also having a much better starting point.

[**TODO:** Current runs: the 2026-10-11 runs (R1 to R7) start the whole output layer of ϕ_{aug} at zero (`G_INIT` zero, D-240), as the authors' public code does (D-237), so $g^{\text{n}}_{\text{aug}} = 0$ and $\bar{x}$ is zero after the first step. In the final runs, the added-state rows of the network's final output layer are randomly initialized (D-248, `FINAL-RUN-FIXES.md` item 6); (26) in Section III-D then changes accordingly. PS with a live start is so far one seed with the ResNet variant (4.0×10^{-6} m against 1.0×10^{-5} m for plain training).]

We therefore propose predictive-state (PS) initialisation, which is a data-driven initialisation for the added states. Similar to W^b after its initialisation is done, the parameters are optimized through (25). PS is adapted from a two-stage regression of predictive-state learning [23, Sec. 2], and it makes use of initialisation before backpropagation through time [24, Sec. 4.2].

The first stage (S1) predicts the residual of the model, instead of features of future observations as in [23]. The second stage (S2) trains the nonlinear state update instead of a linear operator. We continue training V during the first stage, while Downey et al. finish their initialisation before starting BPTT [24, Sec. 4.2]. Fig. 5 shows which samples are used for both stages.

Stage S1 trains the encoder's added-state outputs to predict the residual of the augmented model (13) using the model parameters ϑ from the latest V update step. Each update of V is followed by one S1 update with its own Adam optimiser. For the S1 update we select a minibatch $\mathcal{B}$ of samples τ from the training data. The training data is divided into excitation classes ([**TODO:** see section]), and for classes with at least three records one is left out of S1 and S2 to evaluate ν (28). The left out data is still used in V. For each sample τ, the encoder uses the last $n + 1$ samples up to τ, χ_τ, to estimate the initial state $\hat{x}^{\text{n}}_{\tau|\tau}$. From this state the augmented model is simulated in the closed loop of Section III-C. We calculate

the residual from the next $H = n + 1$ samples, from $\tau + 1$ to $\tau + H$. [**Note:** The current results use a residual target starting at τ, so the encoder and target share sample τ. The final runs will start the target at $\tau + 1$, as described here.] The top of Fig. 5 shows the window χ_τ and the samples of the residual e_τ. We have chosen to use the same number of samples as the encoder window. Then a temporary decoder d with only the added states as inputs $\bar{x}_{\tau|\tau} = \psi_a(\theta_{\text{enc}}, \chi_\tau)$ is used to predict this residual:

$$
\begin{aligned}
e_\tau &= \frac{1}{\sigma_e} \operatorname{col}\left(y^{\text{n}}_k - \hat{y}^{\text{n}}_{k|\tau}\right)_{k=\tau+1}^{\tau+H}, \\
d(\theta_{\text{p}}, \bar{x}) &= W_{\text{p},2} \tanh\left(W_{\text{p},1}\bar{x} + b_{\text{p},1}\right) + b_{\text{p},2}, \\
&\min_{\theta_{\text{p}}, \theta^a_{\text{enc}}} \frac{1}{|\mathcal{B}|\, H\, n_y} \sum_{\tau \in \mathcal{B}} \left\| d\left(\theta_{\text{p}}, \psi_a(\theta_{\text{enc}}, \chi_\tau)\right) - e_\tau \right\|_2^2,
\end{aligned}
\tag{27}
$$

where $\hat{y}^{\text{n}}_{k|\tau}$ is the output from the closed-loop simulation of Section III-C, starting at $\hat{x}^{\text{n}}_{\tau|\tau}$. This simulated output is kept fixed during the S1 update. $\mathcal{B}$ is the minibatch of samples τ and σ_e is the RMS of all residual values in $\mathcal{B}$. The decoder $d : \mathbb{R}^{n_{\bar{x}}} \to \mathbb{R}^{Hn_y}$ has parameters $\theta_{\text{p}} = (W_{\text{p},1}, b_{\text{p},1}, W_{\text{p},2}, b_{\text{p},2})$, with $W_{\text{p},1} \in \mathbb{R}^{n_h \times n_{\bar{x}}}$ and $W_{\text{p},2} \in \mathbb{R}^{Hn_y \times n_h}$. Both $W_{\text{p},2}$ and $b_{\text{p},2}$ are initially set to zero, so that the decoder starts by predicting zero residual. Only θ_{p} and θ^a_{enc} are updated by S1. The parameters of ϑ, including θ^a_{enc} are still trained through V.

The expected squared error is minimised by the conditional mean $\mathbb{E}[e_\tau \mid \chi_\tau]$. The decoder only sees up to $\bar{x}_{\tau|\tau}$, not the full history χ_τ, this forces the encoder to use the added states to predict the future residual. [**TODO:** Instead of using features of future observations as Hefny et al. [23, Sec. 2] for predictive-state learning, we use the model residual.] To avoid correlation between the history and future noise Hefny et al. separate the past and future samples [23, Sec. 2]. We borrow this principle by having the residual start at $\tau + 1$, so that there is no overlap between the samples used for the encoder and to calculate the residual. [**TODO:** Venkatraman et al. use a similar approach by predicting future observations from the internal states during training [25, Sec. 3].] For the PS implementation the decoder predicts the model residual and is removed after S1. [**TODO:** Current runs (`FINAL-RUN-FIXES.md` item 5): the 2026-10-11 runs start the target at τ (`ps2_opening.py`, `_sample` and `_residual`), so sample τ is both the last sample of χ_τ and the first of e_τ, and the noise in y_τ enters both sides of (27).] [**TODO:** Missing: Hefny et al. model the noise as independent per sample (Fig. 1). Our measurement noise is low-pass and fed back by the controller, so disjoint windows remove the shared sample but not all correlation between χ_τ and the noise in e_τ. Candidate: one sentence here or in the Discussion (basis: D-238 noise model).]

To evaluate how well $\bar{x}_{\tau|\tau}$ performs at predicting the residual, a fixed set $\mathcal{C}$ of windows from the held out data is used. For this set the squared prediction error of the decoder divided by the squared residual is computed,

$$
\nu = \frac{\sum_{\tau \in \mathcal{C}} \left\| d\left(\theta_{\text{p}}, \psi_a(\theta_{\text{enc}}, \chi_\tau)\right) - e_\tau \right\|_2^2}{\sum_{\tau \in \mathcal{C}} \left\| e_\tau \right\|_2^2}. \tag{28}
$$

S1 is stopped when a maximum number of updates is reached or when ν stops improving, or at an early check when ν is above $\nu_{\max}$. S2 starts only if ν is below a threshold $\nu_{\max}$, else the decoder is removed and training continues. In both cases the encoder changes from S1 are kept. The threshold rule for S2 is given in Appendix A-E. [**TODO:** Missing: basis of the PS switch and stop rules (S1 plateau and cap, screen update and threshold, S2 plateau and cap, snapshot sizes, calibration split as the last record of each class with three or more records). Candidate: rows of the hyperparameter table in Section V, all HEURISTIC (`PS2Spec`; `PS2-METHOD.md` Sec. 5), noting that the S1 cap, not the plateau, ended S1 in every run so far.]

S1 has only trained the part of the encoder that initializes the added states $\bar{x}_{\tau|\tau}$, $g^{\text{n}}_{\text{aug}}$ is only updated by V. S2 trains the state update itself. In S2 $g^{\text{n}}_{\text{aug}}$ is trained to predict the next added state $\bar{x}_{k+1}$ from the full state $\hat{x}^{\text{n}}_k$ and the input u^{n}_k. For S2 samples k_i from the training data are selected, except the same held-out records from S1 are left out. For each sample the encoder estimates $\hat{x}^{\text{n}}_{k_i|k_i}$ from χ_{k_i} and $\bar{x}_{k_i+1|k_i+1}$ from the next window χ_{k_i+1}. Fig. 5 shows an example with $k_i = \tau$. The window χ_τ at the top is used for $\hat{x}^{\text{n}}_{k_i|k_i}$, and the shifted window at the bottom is used for $\bar{x}_{k_i+1|k_i+1}$. We calculate these pairs once after S1, this set $\mathcal{P}$ is kept fixed.

During S2 the model is not simulated in closed-loop and V is paused. We instead train $g^{\text{n}}_{\text{aug}}$ on $\mathcal{P}$ using one-step regression with the recorded actuator force $u^{\text{n}}_{k_i}$ as input, and a separate Adam optimiser:

$$
\begin{aligned}
\mathcal{P} &= \left\{ (\hat{x}^{\text{n}}_{k_i|k_i},\, u^{\text{n}}_{k_i},\, \bar{x}_{k_i+1|k_i+1}) \right\}_{i=1}^{N_{\mathcal{P}}}, \\
\hat{x}^{\text{n}}_{k_i|k_i} &= \psi(\theta_{\text{enc}}, \chi_{k_i}), \quad \bar{x}_{k_i+1|k_i+1} = \psi_a(\theta_{\text{enc}}, \chi_{k_i+1}), \\
&\min_{\theta_{\text{aug}}} \frac{\sum_{i \in \mathcal{B}_{\mathcal{P}}} \left\| g^{\text{n}}_{\text{aug}}\left(\theta_{\text{aug}}, \hat{x}^{\text{n}}_{k_i|k_i}, u^{\text{n}}_{k_i}\right) - \bar{x}_{k_i+1|k_i+1} \right\|_2^2}{\sum_{i \in \mathcal{B}_{\mathcal{P}}} \sum_{j=1}^{n_{\bar{x}}} \left(\bar{x}_{j,k_i+1|k_i+1} - m_j\right)^2},
\end{aligned}
\tag{29}
$$

where θ_{enc} is not updated during S2, $\mathcal{B}_{\mathcal{P}} \subset \mathcal{P}$ is a minibatch, $\bar{x}_{j,\cdot}$ is the jth added state and m_j is the mean of $\bar{x}_{j,k_i+1|k_i+1}$ over $\mathcal{B}_{\mathcal{P}}$. The cost is the fraction of the variance in the next added states that is not explained by the prediction.

The parameters with the lowest cost on the separate set of pairs from the held-out records are kept. For S2 we use the recorded actuator force $u^{\text{n}}_{k_i}$ applied to the system at k_i. The encoder estimates $\bar{x}_{k_i+1|k_i+1}$ from χ_{k_i+1}, using the data up to $k_i + 1$. At the start of a window in (22) $x^c = 0$, resulting in the model input

$$
\hat{u}_{\text{act},k_i} = u_{\text{act},k_i} + D_{\text{fb}}\left(y_{k_i} - \hat{y}_{k_i|k_i}\right),
$$

when the encoder reproduces y_{k_i} this equals the recorded input, because at this sample only the controller feedthrough D_{fb} acts on the output error. For S2 $g^{\text{n}}_{\text{aug}}$ is therefore trained with approximately the input it receives at that sample in the closed-loop simulation of V. S2 updates all parameters θ_{aug}

of ϕ_{aug}. The cost only depends on the added-state rows, so the physical rows of W_L and b_L are not changed. The hidden layers of ϕ_{aug} are shared by $f^{\text{n}}_{\text{aug}}$ and $g^{\text{n}}_{\text{aug}}$, so S2 also updates $f^{\text{n}}_{\text{aug}}$. [**TODO:** The encoder windows at k_i and k_i+1 share measurement samples, so noise enters both sides of the S2 regression. Hefny et al. discuss how correlated noise can bias such a fit [23, Sec. 2]. We do not correct for this effect in S2.] When S2 is completed the optimization continues minimizing (25)

$$\vartheta_{\text{PS}} = \left(\beta^{(i_1)},\, \theta^{\text{S2}}_{\text{aug}},\, \theta^{(i_1)}_{\text{enc}}\right), \tag{30}$$

followed by the L-BFGS polish and selection of Section III-D. Here $\beta^{(i_1)}$ and $\theta^{(i_1)}_{\text{enc}}$ are the parameters after S1 and $\theta^{\text{S2}}_{\text{aug}}$ contains the parameters after S2. When S2 is skipped, ϑ_{PS} contains the parameters at the end of S1.

PS has only changed the starting point of minimising V with its initialization phases, not what is being minimized. The models with and without PS have the same number of parameters, since the decoder is removed.

IV. Interpretability by Orthogonal Construction

- Goal from Section III: an estimate of θ_{base} that is unique under stated conditions; names IV-A and IV-B (style profile)
- Contribution: the extension Györök 2026 Sec. 6 leaves open: a state-space baseline without full-state measurement, here nonlinear in its parameters and identified in closed loop (this work)

The split between baseline and network in the augmented model of Section III-D is not unique, so the estimate of θ_{base} can lose its physical meaning [11, Sec. 3.2]. Our goal is an estimate that is unique under stated conditions. Györök et al. construct such an estimate for input-output baselines that are linear in their parameters. They name state-space baselines without full-state measurement as future research [11, Sec. 6]. We extend their construction to our baseline, a state-space model that is nonlinear in its parameters. Only its positions are measured. It is identified in closed loop. Section IV-A constructs an orthogonal correction of the physical rows and gives the condition under which its estimate equals the true combinations. Section IV-B states the identification problem of the corrected model.

A. Orthogonal correction of the physical rows

- The non-unique split of Section III lets the network negate the baseline (Kessels Sec. 5.3.1.3); to first order a term along $\partial f^{\text{n}}_{\text{base}}/\partial\beta$ cancels a change of β (this work)
- On the gantry, each direction is an $M(Y)^{-1}$-weighted mass, damping or stiffness force, propagated by RK4 (this work)
- The research plan's penalty trades orthogonality against fit; the construction needs no weight but a baseline linear in its parameters (Györök 2025 Eq. (13), Remark 1; 2026 Sec. 3.2, Eq. (2))
- Regressor: Jacobian of the normalised transition in β at β_{lin}; range of the ten combinations; the offset column Γ omitted, for two reasons (Györök 2025 Sec. 4, Eqs. (15), (17), (18); D-186; D-192; adapted)
- A fixed reference set for the coefficient (Györök 2026 Sec. 3.2, after Eq. (12); adopted); tuples of measured positions, differenced velocities, $\bar{x} = 0$, recorded input, each with a successor (current runs: revnote, `FINAL-RUN-FIXES.md` item 11, D-252); data rather than a baseline simulation: the regressor at measured data as in Györök 2026 Eq. (5), on the system's trajectories, the next states define ε, fixed before training (Györök 2025 Sec. 3; D-254)
- Coefficient on the stack, full column rank, orthogonal corrected stack (Györök 2026 Eqs. (8), (10), (11), Lemma 3, Assumption 1; adapted)
- Correction on the physical rows at every step; added-state rows not corrected; coefficient differentiated (Györök 2026 Eqs. (9), (10), (13); D-190; adapted)
- Linearisation point rebuilt per epoch at a detached β: a fixed objective per epoch, the point follows the estimate (Györök 2025 Sec. 3, Remark 2, and Sec. 4; D-190; D-254; adapted)
- Recovery: true one-step discrepancy (34); to first order the minimiser is $\beta^\star + \mathbf{\Phi}^{+}\varepsilon$ for every network, true iff $\mathbf{\Phi}^{\top}\varepsilon = 0$; depends on the excitation; one-step at $\bar{x} = 0$ (Györök 2026 Cond. 4, Thm. 7, Remark 5; adapted, no exact fit)

- Problem: the split between baseline and network is not unique; the network can cancel the baseline (negation)
- Choice: the orthogonal construction of Györök et al.
- Choice: the baseline is not linear in its parameters, so we use its derivative instead: the sensitivity of the one-step map to the ten trained coordinates β, taken at the current point β_{lin}:

The non-unique split lets the network negate the baseline, Kessels' term for learned terms that partially cancel the first-principles terms [13, Sec. 5.3.1.3]. Let $\beta' \in \mathbb{R}^{10}$ be other values of the training coordinates β of (24). The network term $f^{\text{n}}_{\text{aug}} - (\partial f^{\text{n}}_{\text{base}}/\partial\beta)(\beta'-\beta)$ with β' then gives, to first order in $\beta'-\beta$, the same one-step map as $f^{\text{n}}_{\text{aug}}$ with β. On the gantry, each column of $\partial f^{\text{n}}_{\text{base}}/\partial\beta$ is the RK4 propagation, scaled by T_x, of the acceleration change $-M(Y)^{-1}\left(\frac{\partial M}{\partial\beta_j}\ddot{q} + \frac{\partial C}{\partial\beta_j}\dot{q} + \frac{\partial K}{\partial\beta_j}q\right)$ of (2) at fixed state and input. A negating term therefore acts as an added mass, damping or stiffness force whose effect on the state depends on Y.

The research plan proposed the orthogonality penalty of Györök et al., whose weight trades orthogonality against fit [26, Eq. (13), Remark 1]. We instead adopt their later construction, which makes the network term orthogonal to the baseline without a weight to tune [11, Secs. 3.2, 5.2]. It needs a baseline linear in its parameters [11, Eq. (2)]. Our transition is rational in θ_{base} through $M(Y)^{-1}$, so it has no linear-in-parameters form.

We therefore linearise the normalised transition in its parameters, as Györök et al. do for a nonlinear baseline [26, Sec. 4, Eq. (15)]. We differentiate with respect to β rather

than the fourteen raw parameters, whose four lost directions (Section II-C) would give the Jacobian four null directions. The map (24) has a nonsingular Jacobian for $m_{\Delta,0} \neq 0$, so the Jacobian in β spans the sensitivity directions of the ten combinations θ_{base}. Their extended regressor adds an offset column Γ, which adds the direction of the transition $f^{\text{n}}_{\text{base}}$ itself to the protected range [26, Eqs. (17), (18)]. We omit it, for two reasons. To first order, the estimate of β depends on the data only through their projection onto the range of the stacked Jacobian (below (34)). The offset would therefore constrain directions that do not move the estimate. A parameter derivative also does not change when the origin of the normalised state moves, whereas the offset does, so the protected directions would depend on the centring μ_x of (15). The sensitivity is

$$\Phi(\beta_{\text{lin}}, \tilde{x}^{\text{n}}, u^{\text{n}}) = \left.\frac{\partial f^{\text{n}}_{\text{base}}\big(\theta_{\text{base}}(\beta), \tilde{x}^{\text{n}}, u^{\text{n}}\big)}{\partial \beta}\right|_{\beta=\beta_{\text{lin}}}, \tag{31}$$

- Choice: only the range of Φ is protected, without the offset column Γ of Györök et al.
- Choice: Φ is evaluated on fixed one-step tuples $i \in \mathcal{I}$ from the training data: measured positions, finite difference to get the velocities, added states $\bar{x} = 0$
- The network's physical rows are fitted by the stacked sensitivity in least squares; the remainder is orthogonal to its range:

where $\beta_{\text{lin}} \in \mathbb{R}^{10}$ is the linearisation point and $\Phi : \mathbb{R}^{10} \times \mathbb{R}^6 \times \mathbb{R}^3 \to \mathbb{R}^{6\times 10}$. [**TODO:** Open question for Maarten and Jan: omit or keep the offset column Γ. For omitting: the estimate of β depends on the data only through $\boldsymbol{\Phi}$ (below (34)); the protected directions then do not depend on the centring; in the construction we adopt, the regressor equals the parameter derivative [11, Eq. (39)]; parameter uniqueness needs the rank of $\boldsymbol{\Phi}$ only [11, Assumption 1] (D-186 reasons 1, 2, written for the earlier trajectory basis). For keeping: Györök et al. include it for a nonlinear baseline [26, Eq. (18)]; D-190 records the supervisors' condition as non-overlap with the baseline output, the column span of $[\boldsymbol{\Phi}\ \ \boldsymbol{f}^{\text{n}}_{\text{base}}]$, and measured that the column adds a direction (57 percent of it outside range $\boldsymbol{\Phi}$). Without it, a network term along $\boldsymbol{f}^{\text{n}}_{\text{base}}$ is charged only through its component in range $\boldsymbol{\Phi}$ (D-186 caveat). Candidate: omit (D-192, the current code).]

We evaluate Φ on a fixed reference set, which Györök et al. allow for the construction of the coefficient [11, Sec. 3.2]. Only the positions of the baseline state are measured. Each tuple therefore holds the positions $q_i = P^{-\top} q_{\text{act},i}$ of a training sample, velocities from a fourth-order central difference of q within the record, $\bar{x}_i = 0$ and the recorded input. The tuples cover every sample at which this difference exists, both at the sample and at the next sample. [**Note:** The current results also use the last such sample of each record, which has no successor. The final runs will use the tuples as described here.] Each tuple is one step at the recorded input, so the controller does not enter. Györök et al. simulate the baseline when the states are not measured [26, Sec. 3]. We build the tuples from the data instead, as their later construction evaluates its regressor at the measured data [11, Eq. (5)]. The tuples then lie on the trajectories of the system, where the omitted dynamics act, rather than on those of the baseline at θ^0_{base}. They also hold the measured next states that define the discrepancy (34) below. The tuples are built once, before training, and depend neither on the trained parameters nor on the encoder.

Adapting the coefficient of Györök et al. to these tuples [11, Eqs. (8), (11), Lemma 3], we fit the network's physical rows by the stacked sensitivity in the least-squares sense. As they assume of their regressor [11, Assumption 1], the stack has full column rank:

$$\begin{aligned}
\boldsymbol{\Phi}(\beta_{\text{lin}}) &= \operatorname{col}\big(\Phi(\beta_{\text{lin}}, \tilde{x}^{\text{n}}_i, u^{\text{n}}_i)\big)_{i\in\mathcal{I}},\\
\boldsymbol{f}^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}) &= \operatorname{col}\big(f^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}, \operatorname{col}(\tilde{x}^{\text{n}}_i, 0), u^{\text{n}}_i)\big)_{i\in\mathcal{I}},\\
\theta_{\text{aux}}(\theta_{\text{aug}}) &= \boldsymbol{\Phi}(\beta_{\text{lin}})^{+}\, \boldsymbol{f}^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}),
\end{aligned} \tag{32}$$

so that the corrected stack is orthogonal to the sensitivity:

$$\boldsymbol{\Phi}(\beta_{\text{lin}})^{\top}\big(\boldsymbol{f}^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}) - \boldsymbol{\Phi}(\beta_{\text{lin}})\,\theta_{\text{aux}}(\theta_{\text{aug}})\big) = 0.$$

- Choice: the correction is subtracted from the physical rows at every step; the added-state rows are not corrected:

where $\mathcal{I}$ is the set of tuples, $N = |\mathcal{I}|$, $\tilde{x}^{\text{n}}_i \in \mathbb{R}^6$ and $u^{\text{n}}_i \in \mathbb{R}^3$ are the state and input of tuple i in the coordinates of (15), $\boldsymbol{\Phi}(\beta_{\text{lin}}) \in \mathbb{R}^{6N\times 10}$, $\boldsymbol{f}^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}) \in \mathbb{R}^{6N}$, $^{+}$ is the Moore-Penrose pseudoinverse and $\theta_{\text{aux}} \in \mathbb{R}^{10}$.

As in [11, Eqs. (9), (10), (13)], the coefficient is a function of the network parameters, and every prediction subtracts the sensitivity times the coefficient. We subtract it from the physical rows at every step of the augmented model, at the state and input of that step. The added-state rows are not corrected, because the baseline has no added-state update to be orthogonal to. The correction is

$$f^{\text{n}}_{\text{obc}}(\theta_{\text{aug}}, \tilde{x}^{\text{n}}, u^{\text{n}}) = \Phi(\beta_{\text{lin}}, \tilde{x}^{\text{n}}, u^{\text{n}})\,\theta_{\text{aux}}(\theta_{\text{aug}}), \tag{33}$$

- Choice: the gradient of the training cost V of (25) to the network parameters θ_{aug} includes the path through $\theta_{\text{aux}}(\theta_{\text{aug}}) = \boldsymbol{\Phi}^{+}\boldsymbol{f}^{\text{n}}_{\text{aug}}(\theta_{\text{aug}})$; with θ_{aux} held fixed, each step would give the network the derivative of its uncorrected output
- Choice: β_{lin} is set to the current β at each epoch and held fixed within it

where $f^{\text{n}}_{\text{obc}} : \mathbb{R}^{n_{\text{aug}}} \times \mathbb{R}^6 \times \mathbb{R}^3 \to \mathbb{R}^6$. Differentiating $\theta_{\text{aux}}(\theta_{\text{aug}})$ gives the network, at each state and input, the derivative of the corrected term. A detached coefficient would give that of the uncorrected term. [**TODO:** Missing: the planned figure (header): the network's physical stack split into its part in range $\boldsymbol{\Phi}(\beta_{\text{lin}})$ and the orthogonal part, with an inset showing the corrected physical rows, the uncorrected added-state rows and their route back through $f^{\text{n}}_{\text{aug}}$ one step later. Candidate: none drawn yet.]

We set β_{lin} to the current β at each epoch boundary, rebuild $\boldsymbol{\Phi}(\beta_{\text{lin}})$ there and hold it for the epoch. No gradient therefore reaches β through β_{lin}. Each epoch thus minimises a fixed objective, while the linearisation point follows the estimate

between epochs. For a nonlinear baseline, Györök et al. update this point at every evaluation of the objective or fix it at the nominal parameters [26, Sec. 4]. Our interval adapts the per-epoch recomputation they allow for the state estimates in their linear case [26, Remark 2].

Orthogonality on the tuples makes the estimate of β unique to first order, but not yet equal to the true combinations. That depends on what the baseline misses in the data. Let $\beta^\star \in \mathbb{R}^{10}$ be the coordinates of the true combinations. On the tuples, the true one-step discrepancy is

$$\varepsilon = \tilde{\boldsymbol{x}}^{\mathrm{n}}_{+} - \boldsymbol{f}^{\mathrm{n}}_{\mathrm{base}}(\beta^\star), \tag{34}$$

where $\tilde{\boldsymbol{x}}^{\mathrm{n}}_{+} = \mathrm{col}(\tilde{x}^{\mathrm{n}}_{i+1})_{i\in\mathcal{I}}$ holds the next state of each tuple, $\boldsymbol{f}^{\mathrm{n}}_{\mathrm{base}}(\beta) = \mathrm{col}\big(f^{\mathrm{n}}_{\mathrm{base}}(\theta_{\mathrm{base}}(\beta), \tilde{x}^{\mathrm{n}}_i, u^{\mathrm{n}}_i)\big)_{i\in\mathcal{I}}$ and $\varepsilon \in \mathbb{R}^{6N}$. Since the tuple velocities are differenced positions, ε also holds the error of this reconstruction. We write $\boldsymbol{\Phi}$ for $\boldsymbol{\Phi}(\beta_{\mathrm{lin}})$. To first order, the one-step error of the corrected model on the tuples is then minimised by $\beta = \beta^\star + \boldsymbol{\Phi}^{+}\varepsilon$ for every θ_{aug}. This minimiser equals $\beta^\star$ if and only if $\boldsymbol{\Phi}^{\top}\varepsilon = 0$. We adapt this from the recovery theorem of Györök et al., which in addition assumes that the model reproduces the noise-free system dynamics on the training data exactly [11, Cond. 4, Assumption 6, Thm. 7, Eq. (21)]. Without the correction, the minimiser is $\beta^\star + \boldsymbol{\Phi}^{+}(\varepsilon - \boldsymbol{f}^{\mathrm{n}}_{\mathrm{aug}})$, which moves with the network. The condition $\boldsymbol{\Phi}^{\top}\varepsilon = 0$ depends on the excitation. Györök et al. note that an input can be designed to meet it when the structure of the omitted terms is known [11, Remark 5]. The result concerns the one-step error on the tuples at $\bar{x} = 0$, whereas (35) minimises a multistep closed-loop output error. It is therefore a surrogate, not a statement on the trained model.

B. Identification of the corrected model

- Need: the identification problem of Section III-D must include the correction (33); trained parameters, coordinates and cost stay (2026 Eq. (22); code)
- Problem display: cost and changed physical-row line, the other constraints by number; β_{lin} held fixed and refreshed per epoch, θ_{aux} re-solved at every evaluation (D-190 items 5, 6; code)
- Per phase: S1 fits the corrected residual; S2 does not see the correction but changes θ_{aug}; Adam updates; validation uses the last epoch's basis; selection rebuilds the basis; the polish keeps it (D-198; code)
- The polish is compared with the validation error after the basis rebuild; current runs: compared with the pre-rebuild score (revnote; `FINAL-RUN-FIXES.md` item 12, D-253)
- Current runs: the polish is also gated on the error of the combinations against the truth, an oracle (revnote; `FINAL-RUN-FIXES.md` item 2)
- $\mathcal{M}_{\mathrm{OBC}}$ named, input and output as in Section III; compared with $\mathcal{M}_{\mathrm{PS}}$ in Results (THESIS-RESULTS steps 5, 6; notation (?))
- The identification problem of Section III-D gets the correction in the physical-row update; trained parameters and cost are unchanged:

The identification problem of Section III-D must now include the correction in the physical-row update. The trained parameters $\vartheta = (\beta, \theta_{\mathrm{aug}}, \theta_{\mathrm{enc}})$ and the cost V are unchanged:

$$\begin{aligned}&\min_{\vartheta}\ V(\vartheta) \quad \text{s.t.}\\ &\tilde{x}^{\mathrm{n}}_{k+1|\tau} = f^{\mathrm{n}}_{\mathrm{base}}\big(\theta_{\mathrm{base}}(\beta), \tilde{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big) + f^{\mathrm{n}}_{\mathrm{aug}}\big(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big)\\ &\qquad\qquad - f^{\mathrm{n}}_{\mathrm{obc}}\big(\theta_{\mathrm{aug}}, \tilde{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\big),\\ &\text{the other constraints of (25),}\end{aligned} \tag{35}$$

- θ_{aux} is solved again at every evaluation of V; β_{lin} is refreshed per epoch
- The PS stages, optimiser and selection rule are reused; $\mathcal{M}_{\mathrm{OBC}}$ is $\mathcal{M}_{\mathrm{PS}}$ with the correction

where β_{lin} in (33) is held fixed during each update and refreshed per epoch (Section IV-A), and θ_{aux} is solved by (32) from the current θ_{aug} at every evaluation of V.

The optimiser, the stages of Section III-E and the selection rule of Section III-D are reused. Each S1 update fits the residual of the corrected model. It changes only θ_{p} and $\theta^{a}_{\mathrm{enc}}$. S2 fits $g^{\mathrm{n}}_{\mathrm{aug}}$, which the correction does not touch. It changes θ_{aug}, so θ_{aux} changes at the next evaluation of V. The Adam updates of V, during S1 and in S3, change every entry of ϑ. Validation uses the basis of the last epoch. After the best Adam checkpoint is restored, the basis is rebuilt at its β. The polish then changes every entry of ϑ with this basis, under the acceptance rule of Section III-D. The polish is compared with the validation error of the model with the rebuilt basis. [**Note:** The current results compare the polish with the validation error measured before the basis is rebuilt. The final runs will measure it after the rebuild, as described here.] [**Note:** The current results also reject the polish when the error of the ten combinations against the true parameters of the simulated system rises by more than 0.1 %. The final runs will accept the polish on the validation error alone, as described in Section III-D.]

We call $\mathcal{M}_{\mathrm{OBC}}$ the model $\mathcal{M}_{\mathrm{PS}}$ with the correction (33), identified by (35). Its input is the actuator force u_{act} and its output the actuator positions q_{act}. It starts as $\mathcal{M}_{\mathrm{PS}}$ does, with the first basis built at $\beta = 0$. Section VI compares $\mathcal{M}_{\mathrm{OBC}}$ with $\mathcal{M}_{\mathrm{PS}}$ on the ten combinations and on accuracy. [**TODO:** Missing: model notation. Candidate: the calligraphic $\mathcal{M}_{\mathrm{OBC}}$, since plain M clashes with $M(Y)$ (README open conflict on model names).]

V. Experiment Design

A. System and data

- System: LPV baseline plus a payload absorber (50 % of the head mass, 150 Hz, $\zeta = 0.03$) plus tanh Coulomb rail friction, in closed loop with controller K1 (Fig. 6)
- Choice: absorber and friction only in the system, as known missing dynamics
- Choice: output noise at the measured machine level; noise floor = noisy record minus its noise-free twin
- Excitation: multisine of 106 to 297 Hz

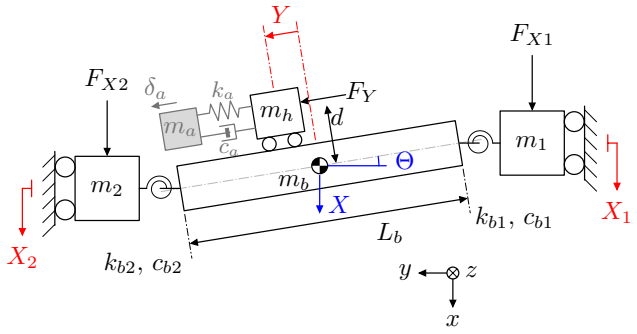


Fig. 6: Simulated gantry with a payload-mounted absorber. The light-gray elements $(m_a, k_a, c_a, \delta_a)$ are absent from the baseline.

- Types of motion: standstill at a fixed Y, slow Y sweeps, X-Y Lissajous paths, S-curve point-to-point moves, sine-shaped
- Training (18 records): fits the model; Y within ± 0.30 m, accelerations up to 75 % of the machine maximum
- Validation (6 records): only selects the checkpoint and the black-box size; never reported
- Test (12 records): every reported number
- Interpolation (I1 to I6): inside the training range, with new multisine phases, new Y positions and new move sequences (sine and S-curve routes, X-only and Y-only moves, long strokes)
- Extrapolation (E1 to E6): each differs from its I record in one property outside the training range: Y position (E1, E2, up to 0.39 m), acceleration (E3, 100 %), velocity (E4, 1.95 m/s), setpoint using movement from machine (E6), a changed controller (E5, not done currently)

B. Compared models

- Baseline: LPV at the true and at the detuned parameters; LTI frozen at $Y = 0$
- Augmented: static ($n_a = 0$), plain training, PS initialisation ($n_a = 2$ and 8), unconstrained against OBC
- Choice: start 10 % detuned for the combinations, the same direction in every run
- Choice: black box = SUBNET on the closed loop $(r, f) \to e$, size selected from 11 configurations on validation

C. Training and evaluation

- Closed-loop training, window 0.1 s, encoder history 29 samples, network 2×16 tanh
- Choice: one seed per run (compute limit)
- Metric: simulation error RMS per axis in m, over the whole test record
- Validation only selects checkpoints; every reported number is on the test set

[**TODO:** Define the two overlaps of Table II (moved from the removed Section IV-C). On one-step tuples of I1 to I6 at $\bar{x} = 0$, with Π the orthogonal projector onto range $\mathbf{\Phi}$ of (32): learned-term overlap $\left\|\Pi \boldsymbol{f}^{\mathrm{n}}_{\mathrm{aug}}\right\|_2 / \left\|\boldsymbol{f}^{\mathrm{n}}_{\mathrm{aug}}\right\|_2$, with $\mathbf{\Phi}$ at the trained coordinates and, for OBC, $\boldsymbol{f}^{\mathrm{n}}_{\mathrm{aug}}$ after the correction (33); missing-dynamics overlap $\|\Pi\varepsilon\|_2 / \|\varepsilon\|_2$, $\varepsilon = \tilde{\boldsymbol{x}}^{\mathrm{n}}_{+} - \boldsymbol{f}^{\mathrm{n}}_{\mathrm{base}}(\beta^\star)$, with $\mathbf{\Phi}$ at the true coordinates $\beta^\star$ and $\tilde{\boldsymbol{x}}^{\mathrm{n}}_{+}$ the next states of the tuples.]

- Generate closed-loop records from the physics baseline plus a truth-only flexible absorber, retaining exact states solely for diagnostics
- Use multisine excitation that covers the absorber pole and anti-resonance while respecting position, velocity, yaw, and force limits
- Assign independent random multisine phases to training, validation, and test records so equal spectra do not duplicate trajectories
- Include multisine-free cycloidal motion profiles matching the measured machine motion to test the operational distribution shift
- Use only the corrected all-channel Coulomb dataset matched to the frictionless excitation; the earlier mismatched-amplitude merge is superseded
- The set-valued friction law requires a velocity threshold below which a rail is treated as stuck. This is set to the Stribeck velocity, $v_s = 2.25 \cdot 10^{-3}$ m/s, taken from Lee et al. [27], Table 5, identified on a ball-screw feed drive with a THK SSR20XW guide (Sec. 5.1). Stribeck velocities reported in the literature span 10^{-5} to 10^{-2} m/s [27, Sec. 5.2], so the chosen value sits mid-range [**TODO:** Conflicts with D-204, where the stick band is `V_EPS = (cc1+cc2)/m_total*ts` labelled THEORY: leine1998, about $3.3 \cdot 10^{-5}$ m/s before the 3 to 10x margin. Confirm which threshold the reported runs use, and whether the second citation (Stribeck range) is the same paper.]
- State that output noise is currently disabled; add it only if a level and measurement location can be estimated from Jasper, ILC data, or stationary records
- State the sample rates, record lengths, number of windows, and total independent motion time rather than quoting overlapping-window count as data volume
- Decision (RK4 value, moved here from Section II): one reproducibility sentence giving the RK4 step, equal to the sampling period after downsampling the 20 kHz records to 4 kHz, with one substep (entry script: `fs_new=4000`, `up_sample=1`). Confirm against the `config.json` of each reported run, since earlier checkpoints may have used another `up_sample`
- Figure (reuse, no separate asset): let the rebuilt training-objective time strip also identify independent phases, short training windows, encoder history, and the long evaluation horizon

[**TODO:** Is the hidden linear absorber sufficient simulation complexity, or is a truth-only cubic spring required to demonstrate nonlinear augmentation?] [**TODO:** discuss noise model and how we filter noise in the setup]

D. Real Telica data and split

- Confirm with Jasper and Dragan which reconstructed ETEL setpoint-generator or ILC-reference motions are operationally representative, including minimum/maximum move, long stroke, acceleration, and dwell
- Use the supervisor-defined split of 11 training, two validation, and two test operating points at 20 kHz
- Convert rail positions and currents to logical metres and Newtons; form velocity states by central differences because direct velocity is not logged
- Keep all iterations at one operating point in the same split to prevent identical reference trajectories leaking across sets
- Describe the X and Y operating-point grid and show whether training covers the interpolation region occupied by both test points
- Use iteration 0 for controller checks and later ILC iterations for distinct feedforward conditions without treating them as independent trajectories
- Mark real-data training and held-out evaluation as planned until a final reproducible run and force-calibration convention exist
- Figure (new): 2D operating-point grid with train, validation, and test locations, operating-range boundary, and trajectory classes; add a 3D or pairwise state-input coverage view only if it reveals coverage hidden by the 2D map

[**TODO:** Freeze the approved operational trajectory set with Jasper and Dragan; should direct drive velocity be requested, and does a third coverage coordinate add information beyond the 2D grid?]

E. Training protocol, comparators, and metrics

- Match records, neural seeds, windows, optimiser budget, and checkpoint criterion across baseline-only, unconstrained, tangent-OBC, affine-OBC, and black-box arms; treat multisine phase seeds separately
- Use physical RMS as the primary candidate metric for tracking and a fixed 200–400 ms post-move settling interval; use NRMS, BFR, or spectra only for a distinct supporting question
- Report physical combination error, projection overlap, added-state ablations, and encoder-init sensitivity alongside predictive metrics
- Values referenced from Section III (moved 2026-10-06), each with an honest basis (derived, inherited, tuned, compute-limited): encoder history n, window length n_f and stride, number of additional states $n_{\bar{x}}$, network width and depth, model rate
- Compared model (moved from Section III-A, 2026-10-11): for $n_{\bar{x}} = 0$ the neural network only corrects the six baseline states, giving the static parallel augmentation of [8, Table 1]; used in the $n_{\bar{x}}$ comparison (`THESIS-RESULTS.md` step 4, run R6)
- Free-run validation (referenced from Section III-C): whole records, encoder at the record start and $x^c = 0$ once, so model error accumulates over the record unlike in a training window; state the record-length-weighted RMS used for checkpoint selection
- Decision (moved from Section II-C): joint-estimation arms start from the nominal φ of the reference implementation detuned by 10% per combination (`combo_init_detune`, HEURISTIC, D-191), with the nominal-parameter prior disabled (D-193); state it as a run setting with its reason
- A parameter prior bounds the drift of the combinations but does not remove the negation flat direction of Section IV; it is off in every arm (D-193)
- Practical identifiability (referenced from Section II-C): report the rank of the stacked one-step transition Jacobian with respect to φ on the training data (rank 10, nullity 4 in θ per the reduced-block docstring), from a saved artefact rather than the docstring
- Scope (from Section II-C): the structural result concerns the ideal, noise-free baseline model; whether the closed-loop records excite all ten combinations is checked here; the augmented model, where the network can absorb part of the baseline, is treated in Section IV; the encoder does not affect the result, as it only estimates the initial state of a simulation, not the model
- Value (moved from Section II-C): the margin $\rho = 0.99$ in (24), a HEURISTIC margin inside the admissible region (D-229)
- Metric (moved from Section II-C): relative combination error in θ_{base}, with the normalisation of the small signed m_Δ stated explicitly
- State per arm whether raw or reduced coordinates are trained; exact-model parameter recovery is an internal implementation check, not a result (moved from Section II-C)
- Give the black-box model validation-selected capacity and horizon so it is a capability comparison rather than an artificially weakened baseline
- Evaluate held-out position/phase interpolation, ASMPT-relevant multisine-free motion, controller transfer, and operating-range extrapolation as separate rungs
- Predeclare expectation, acceptance criterion, verdict, and mechanism for every result rather than presenting isolated scores

[**TODO:** Decide the primary metric, credible controller-transfer arm, and feasible neural-seed count; do not pool initialization, noise, phase, and operating-point uncertainty.]

F. Reproducibility and implementation checks

- Verify controller reconstruction, coordinate transforms, normalisation, RK4 transition, and LFR equivalence before training
- Use the same controller, data conversion, and rollout entry point for training, validation, and checkpoint selection
- Controller rate (moved from Section III-C, 2026-10-06): the records are generated with the controller at 20 kHz, the training loop re-discretises it at the 4 kHz model rate

(D-141); the residual form of Section III-C equals the shared-reference loop only for a compatible controller, signals and initial state
- Compatibility check for that assumption: run the re-discretised controller on the recorded $r - y$ and compare its feedback force with the recorded one; a rollout with $\hat{y} = y$ gives $\hat{u} = u$ by construction and checks nothing
- Controller mismatch: the records use the controller at 20 kHz, training re-discretises it at 4 kHz, so the residual form of Section III-C is approximate and part of the measurement noise is not removed [**TODO:** Missing: size of the controller mismatch. The records use the controller at 20 kHz, training re-discretises it at 4 kHz. Candidate: the compatibility check of the Setup bullet, reported as the force difference against the recorded u_{act}.]
- Record configuration, software precision, seed, dataset identifier, and checkpoint provenance for every reported arm
- Keep oracle states and truth parameters out of the trainable pipeline and label every diagnostic that uses them
- Figure (existing, appendix candidate): use `controller-chain-v1` only as an audit trail for controller reconstruction and discretisation, not as a main method figure

[**TODO:** Which implementation checks belong in the main paper, and which should move to the appendix or repository documentation?]

VI. Results

A. Baseline form

- Criterion: LPV error flat over Y, LTI error growing away from $Y = 0$ (Fig. 7)
- On X the LTI error is 1.5 to 3.7 times the LPV error, growing with $|Y|$; on Y both are equal
- Y scheduling is needed for the X axes

- Expectation and criterion: the LPV-LFR and original physics equations agree numerically across the payload range before model-error comparisons are meaningful
- Verdict and reason: report inertia invertibility, LFR/RK4 equivalence, and the remaining baseline error caused by the truth-only absorber
- Figure: baseline and oracle output errors versus time for representative payload positions; supports the claim that the omitted mode is systematic and position dependent
- Table: baseline free-run physical RMS for LPV and frozen-LTI models across held-out Y; tests the research plan's Aspect 1 rather than assuming LPV superiority
- Expectation and criterion: direct and residual closed-loop forms match at numerical precision when controller assumptions hold
- Evidence for Section III-C (moved 2026-10-06): on the thesis data, open-loop rollouts accumulate position error on the free axes X and Y, closed-loop rollouts do not; re-measure, do not reuse the older D-142 numbers

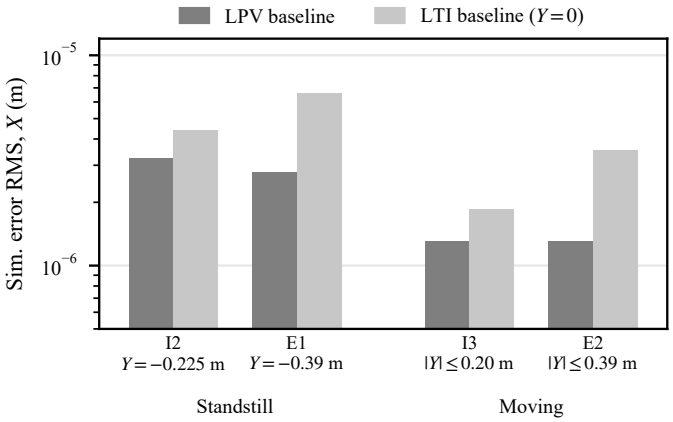


Fig. 7: Simulation error RMS on the X axes of the LPV baseline and the LTI baseline at $Y = 0$, both at the true parameters.

- Figure: recorded input, controller correction, and model error over one rollout; supports the interpretation that feedback can suppress visible output error while the open-loop model remains wrong

[**TODO:** The LPV-versus-frozen-LTI comparison is required by the plan but may not yet have a final matched result; run it or narrow Aspect 1 explicitly.]

B. Augmentation structure

- Criterion: error below the baseline and below the model without added states (Table I)
- PS initialisation lowers the Y error about 9 times; plain training and $n_a = 0$ stay near the baseline
- The error in the multisine band disappears with PS initialisation, not with plain training (Fig. 8)
- The learned pole is at 212 Hz, the absorber pole of the system at 212 Hz; $n_a = 8$ adds nothing
- Without added states ($n_a = 0$) the Y error stays at the baseline level
- Time trace of the augmented model and the system (Fig. 9)

- Expectation and criterion: augmentation must reduce held-out long-horizon error, not only the short training loss, relative to the same encoder-initialised baseline
- Verdict and reason: report the best validated improvement and the gap between window fit and free-run accuracy caused by integrated velocity bias
- Figure: validation RMS against optimiser updates for baseline and augmented arms, with window and 12 s metrics; supports or rejects objective alignment
- Figure: error spectra before and after augmentation with absorber pole and anti-resonance marked; tests whether the learned correction targets the missing band
- Figure: clamp each additional state and the full added-state vector to zero during rollout; quantifies what the trained latent dynamics add or harm

TABLE I: Simulation error RMS per axis, mean over test records I1 to I6.

Model	X_1 (m)	X_2 (m)	Y (m)
Baseline, true θ	$1.64 \cdot 10^{-6}$	$1.97 \cdot 10^{-6}$	$1.31 \cdot 10^{-5}$
Baseline, detuned θ	$2.99 \cdot 10^{-6}$	$2.28 \cdot 10^{-6}$	$1.56 \cdot 10^{-5}$
Static parallel, n_a=0	$1.33 \cdot 10^{-6}$	$1.42 \cdot 10^{-6}$	$1.24 \cdot 10^{-5}$
Plain training, n_a=2	$1.18 \cdot 10^{-6}$	$1.26 \cdot 10^{-6}$	$1.06 \cdot 10^{-5}$
PS init., n_a=2	$1.09 \cdot 10^{-6}$	$1.21 \cdot 10^{-6}$	$1.39 \cdot 10^{-6}$
PS init., baseline part only	$1.94 \cdot 10^{-6}$	$1.77 \cdot 10^{-6}$	$1.35 \cdot 10^{-5}$
PS init., n_a=8	$1.04 \cdot 10^{-6}$	$1.19 \cdot 10^{-6}$	$1.44 \cdot 10^{-6}$
PS init., detuned θ	$1.08 \cdot 10^{-6}$	$1.20 \cdot 10^{-6}$	$1.44 \cdot 10^{-6}$
Augmented (OBC), detuned θ	$1.14 \cdot 10^{-6}$	$1.29 \cdot 10^{-6}$	$1.49 \cdot 10^{-6}$
Black-box model	$1.85 \cdot 10^{-5}$	$2.11 \cdot 10^{-5}$	$2.33 \cdot 10^{-5}$
Noise floor	$9.00 \cdot 10^{-9}$	$9.63 \cdot 10^{-9}$	$3.19 \cdot 10^{-9}$

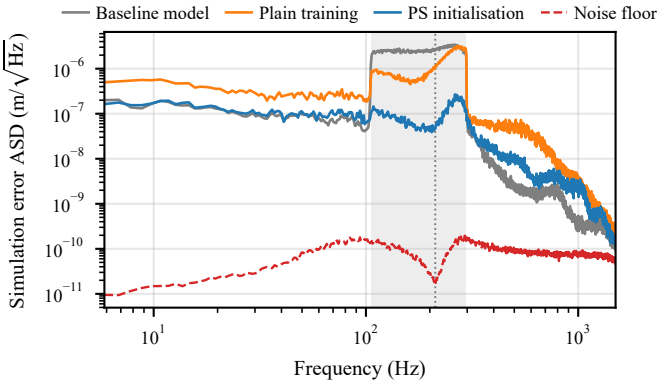


Fig. 8: Amplitude spectral density of the Y simulation error on test records I1 and I2. Shaded: multisine band. Dotted: absorber pole.

- Table: standard model, no-added-state ablation, zero-initialised latent path, and longer encoder history; connects structure changes to predictive benefit
- State the observed weak or harmful added-state contribution as a result if the final run does not reverse it, rather than inferring usefulness from state magnitude

[**TODO:** Are state Gramians useful for explaining each added state's contribution, or would intervention-based ablation be more direct and less linearisation-dependent?]

C. Interpretability

- Criterion: OBC drifts less than the unconstrained arm at equal test RMS (Table II)
- Not met: the parameters drift in both arms, OBC not less
- OBC keeps the learned term out of the parameter directions: overlap 0.12 to 0.13 against 0.91 to 0.99
- Cause: 59 % of the missing dynamics lies in the parameter directions

- Evaluation on the validation-selected best checkpoints of R1/R2 (OBC, detuned/correct start) and R4/R5 (unconstrained, detuned/correct start): the following diagnostics require no additional training runs; all reported predictive scores use the test set.

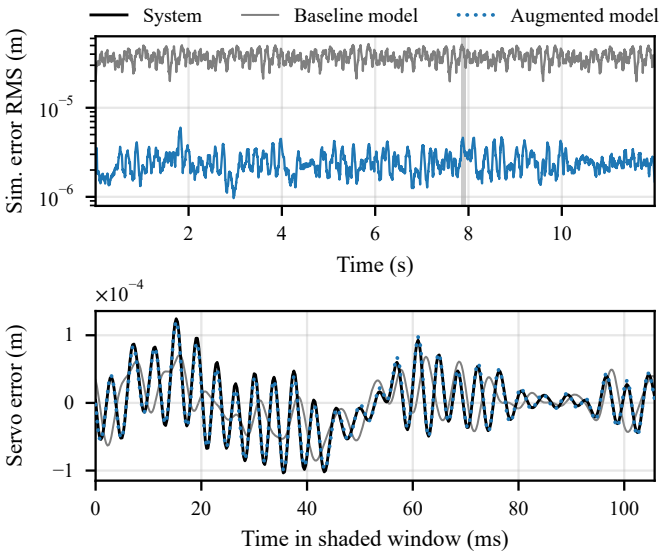


Fig. 9: Test record I1, Y axis. Top: moving RMS of the simulation error. Bottom: servo error in the shaded window.

- Report normalised inner products with the baseline sensitivity space and the baseline-output overlap ρ_f (Eq. (**??**)), both on the $x_a = 0$ reference slice and with active added states along held-out test rollouts. Use the same normalised coordinates and stacking convention, and distinguish reference-slice orthogonality from observed overlap away from that slice.
- Report the ten physical combination errors relative to truth, their aggregate relative-error norm, and displacement from each start; compare OBC with unconstrained estimation for both starts.
- Report full augmented-model and standalone-baseline test prediction RMS using each checkpoint's fitted physical parameters, against the per-record noise floor, to assess physical-model usefulness alongside predictive accuracy.
- Interpret overlap and agreement between the two starts as empirical diagnostics of the decomposition. They do not prove uniqueness or true-parameter recovery; any uniqueness statement must retain the assumptions and scope of the derivation in Section IV-A.
- Expectation and criterion: OBC should reduce physical combination error without materially worsening held-out predictive fit relative to unconstrained joint estimation
- Verdict and reason: the completed arms reach 7.20% no projection, 7.69% tangent OBC, and 8.47% unfinished affine OBC from a common 9.49% start, so aggregate recovery does not establish OBC superiority
- Figure: validation fit versus combination error over training for all three arms; supports the claim that near-equal fit permits materially different physical parameters
- Figure: per-parameter error at matched budget; shows which combinations projection protects and which near-flat directions redistribute error
- Table: basis rank, condition, overlap, final and minimum

TABLE II: Relative error of the ten combinations, unconstrained and OBC, from the detuned and the correct start.

	Detuned start			Correct start	
Rel. error (%)	Start	Unconstr.	OBC	Unconstr.	OBC
$k_{b,\Sigma}$	−10.0	−2.3	−6.2	−9.2	−2.7
c_{g1}	+10.0	+35.1	+45.0	+16.4	+28.3
c_{g2}	+10.0	+44.7	+44.1	+29.4	+35.3
c_y	+10.0	+41.5	+12.6	+30.0	−6.0
$c_{b,\Sigma}$	−10.0	+15.2	+19.9	+28.8	+34.7
m_h	−10.0	−3.3	−3.6	−2.2	−2.2
m_Σ	+10.0	+2.4	+2.5	+3.1	+2.0
m_Δ	+10.0	+33.0	+44.5	+14.4	+34.5
J_{eff}	−10.0	−0.8	−1.9	+3.8	+2.6
d	−10.0	−3.2	+10.4	−9.0	+14.9
Norm	31.6	79.4	81.7	57.1	68.7
Test RMS ($\times 10^{-6}$ m)		1.24	1.31	1.23	1.30
Learned-term overlap		0.99	0.12	0.91	0.13
Missing-dynamics overlap			0.59		
Learned pole (Hz)		212	213	212	214
Learned damping ζ		0.049	0.052	0.051	0.052

parameter error, and validation RMS; links algorithm health to the recovery verdict
- Figure: ρ^* and predicted versus observed parameter bias by singular direction; supports the conditional result that true recovery is limited by non-orthogonal missing physics
- Explain why the 4.96% predicted floor accounts for part of the residual squared error but does not match the observed per-parameter pattern or prove convergence

[**TODO:** Should the incomplete affine arm be retained, rerun to a matched budget, or reported only as exploratory evidence?]

D. Generalisation

- Criterion: at least as good as the black-box model on I, better on E (Fig. 10)
- Met: the augmented model is below the detuned baseline on all records, and 10 to 30 times below the black-box model on I and E
- During motion about 10^{-6} m remains, 100 times above the noise floor
- Expectation and criterion: improvement must progress from held-out position/phase interpolation to ASMPT-relevant motion, controller transfer, and quantified operating-range extrapolation without retuning
- Verdict and reason: separate completed simulation evidence from pending Telica results and explain any failure by excitation coverage, initialisation, or objective mismatch
- Figure: tracking overview plus a dedicated 200–400 ms settling zoom for baseline, augmented model, and black box on an approved operational trajectory
- Table: primary RMS, fixed-window settling RMS, parameter availability, and training data for baseline, augmented, and black-box models on every completed validation rung
- Figure or table: train with one controller and evaluate the frozen models with a credible changed controller; pair this with a plant-domain check for feedback masking
- Figure: error versus multisine amplitude, stroke, acceleration, scheduling position, or state-input distance from the training set; quantify extrapolation rather than calling all held-out data extrapolation
- Report the black box's early improvement and later free-run degradation as a negative comparator result if no stable matched run supersedes it
- Aggregate the final learned-model comparison across neural initialization seeds and show a representative, not best, time trace; report phase realizations separately

[**TODO:** Freeze which validation rungs are complete enough for a final claim; omit rather than speculate about unfinished controller, noise, or hardware experiments.]

VII. Discussion

A. Answers to the four research aspects

- Aspect 1: state whether explicit Y scheduling earns measurable predictive value over frozen LTI baselines
- Aspect 2: distinguish the expressive value of dynamic augmentation from the observed trainability of its added states
- Aspect 3: conclude that OBC enforces a unique local split but true parameter recovery is conditional on the real discrepancy's orthogonality and optimiser convergence
- Aspect 4: compare simulation interpolation, motion-profile transfer, amplitude extrapolation, real-data evidence, and the black-box result without merging them into one score
- Treat controller transfer as the industrial digital-twin use case and as a closed-loop distribution shift distinct from operating-range extrapolation
- Interpretability matters when physical parameters support controller redesign or health monitoring, but its practical value must be tied to a named downstream decision
- Figure plan: introduce no new figure; interpret the architecture, OBC, and result figures already presented instead of adding a summary graphic

[**TODO:** Which result is the primary answer if predictive accuracy and parameter recovery favour different arms?]

B. Limitations and lessons

- Closed-loop feedback improves numerical trainability but can hide plant mismatch, suppress orthogonality overlap, and reward a model that is poor in open loop
- Short windows underweight velocity DC bias on free integrators; varying encoder and rollout length is therefore a structural sensitivity test, not routine tuning
- One strengthened absorber, currently noiseless simulation, a finite neural-seed campaign, and truth-only dry friction bound claims about nonlinear, noisy, and dissipative missing dynamics
- State the final seed count and corrected-friction evidence here; do not retain development-era caveats after the final campaign changes them
- Central-difference velocities, force calibration, and the train-validation-test coverage grid bound the credibility of any real-data conclusion

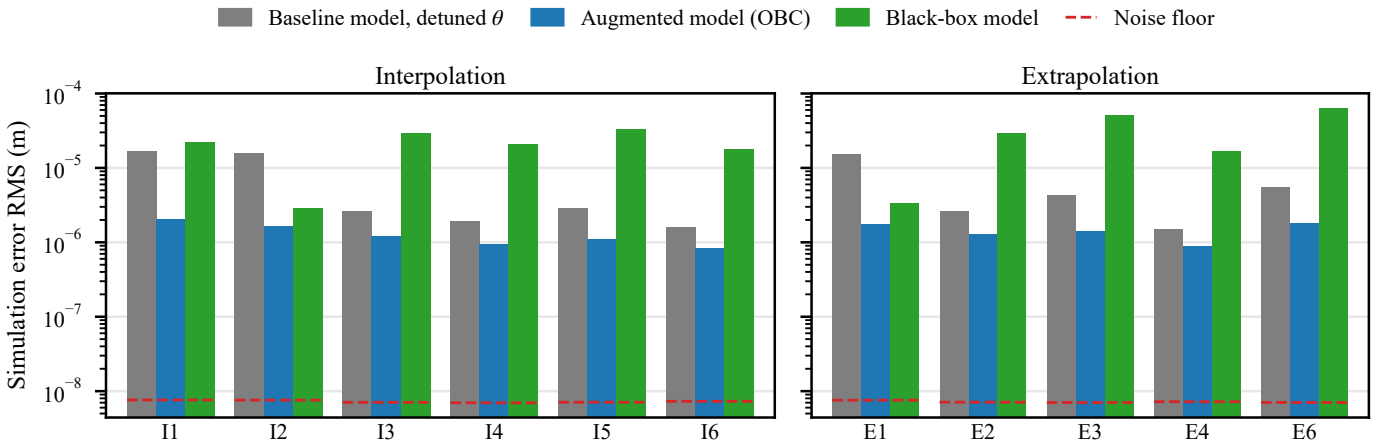


Fig. 10: Simulation error RMS per test record, mean over X_1, X_2, Y.

- The research plan expected validated real-machine generalisation; any missing Telica result is a departure to disclose, not replace with design documentation
- Augmentation versus residual-FRF fitting remains a trade between trajectory-domain nonlinear prediction and a simpler frequency-domain account of missing dynamics
- Pointwise orthogonality is impossible for any nonzero missing force on this gantry: the parameter sensitivity of the three velocity rows has rank 3 at every reference sample (worst $\sigma_3/\sigma_1 = 5.7 \cdot 10^{-4}$, earlier absorber dataset), so $J^\top \Delta^* = 0$ can only arise from cancellation over the record (Section IV-C)
- Coulomb friction in the truth violates the recovery condition by construction: the c_{g1} sensitivity is proportional to $\dot{X}_1$ and the friction is $-c_{c1}\,\mathrm{sign}(\dot{X}_1)$, so their inner product is $-c_{c1}\sum|\dot{X}_1| \neq 0$ for any moving record (D-204); on T_AF, OBC gives a unique split, not unbiased combinations

[**TODO:** Should a cubic-spring benchmark, noise study, and Monte Carlo analysis be completed, or should the thesis explicitly narrow its claims to the measured linear absorber case?] [**TODO:** Missing: the criterion (25) weights each axis by $1/\sigma^2_{y,i}$ (normalised outputs, as [8, Sec. 5.3]), while checkpoint selection and the reported errors are in metres, so training and evaluation weigh the axes differently. Candidate: one limitation sentence, after checking whether Hoekstra et al. (Sec. 6) and Beintema et al. report normalised (NRMS) or physical-unit errors (not yet read).]

C. Implications and future work

- Design excitation so $J^\top \Delta^* = 0$ when true parameter recovery, rather than only a unique decomposition, is required
- Separate learning rates for physical and neural parameters and compare convergence against the predicted nonzero OBC bias floor
- Replace the short-horizon objective or couple windows so integrated velocity bias is visible during training
- Repair and rerun the Coulomb benchmark before claiming that the method learns a legitimate nonzero-mean residual
- Extend evaluation beyond the multisine range and use machine-relevant trajectories to delimit safe interpolation and extrapolation

[**TODO:** Which future-work item is a prerequisite for the thesis claim and which genuinely belongs after graduation?]

VIII. Conclusions and Discussion

A. Conclusions

- Aspect 1: LPV scheduling lowers the error on X, increasingly away from $Y = 0$
- Aspect 2: added states with PS initialisation learn the missing mode; plain training does not
- Aspect 3: OBC keeps the learned term out of the parameter directions; the parameters still drift
- Aspect 4: better than the black-box model on interpolation and extrapolation
- Black box currently doesnt perform well on test sets.
- Recommendation?
- Answer whether LPV-LFR dynamic augmentation improves dual-gantry settling prediction over the physics baseline and black-box comparator
- State whether the learned additional states contributed measurable missing dynamics under the final validated configuration
- Conclude that state-level OBC prevents local tangent overlap by construction, while true parameters are recovered only when the physical discrepancy is orthogonal on the chosen excitation
- Give the final quantitative trade between prediction error and physical combination error on held-out data
- State whether the evidence covers relevant ASMPT motion, settling, controller transfer, or only a narrower simulation validation ladder
- Close with the practical recommendation for when ASMPT should use the augmented model and when the baseline or a simpler residual model is safer

- Figure plan: no conclusion figure; reserve the space for the final quantitative answer and limitations

[**TODO:** Rewrite every conclusion bullet after freezing the final result table; no planned or diagnostic result may appear as achieved.]

B. Discussion

- Why the parameters drift despite OBC (missing-dynamics overlap)
- What the black-box comparison does and does not show
- The error during motion is not captured, likely friction??
- Single seed, simulation only, no controller transfer
- OBC holds only at $\bar{x} = 0$; current definition doesn't take into account the additional states entering the output the next update step.
- Need to fix: current encoder is initialized with true parameters
- Current runs only: the encoder window includes τ, so the first output scored by (25) is one the encoder has seen; the final runs use the window up to $\tau - 1$ (D-250) and the data-based W^b (D-251)
- Future work: friction, excitation for parameter recovery, real data
- Why all states routing.
- why not use the encoder for the orthogonality each epoch instead of finite difference
- why not use the positions since we measure them –¿ train nn and encoder in combination? not full answer
- black box not fair comparison, no init (BLA) no controller

Thesis

Make a new folder for figures and code of the figures for the paper, and the final dataset that will be used.

* laten zien met parameters samenvoegen huidige methode niet correct ik moet iets van jacobian of iets laten zien * ik heb nog steeds niet netjes uitgewerkt voor LPV-LFR met hoe ik het heb omgeschreven die n m ik kan niet zo goed zeggen hoe het heet * verbeterde model met additional states laten zien als ik ze weg haal wat ze toevoegen = op nul zetten. ik weet niet of andere phase multisine validation en test genoeg laat zien hoe goed het werkt. * Duidelijk verantwoorden controller. vooral met integrator problem van de plant en dus X en Y geen stijfheid K = 0 daar. kleine integratie error propegeert gwn door, dit zei maarten ook denk dat dit de error is. * velocity central difference, why not get direct? * Zijn Gramians nuttig om per state te laten zien? * Multisine phase uitleggen kunnen we ook bronnen voor gebruiken. dat verschillende train validation en test verschillende multisine phase hebben. * Met alleen simulatie is huidige simulatie niet genoeg, cubic spring toevoegen?? * benefit van mijn closed-loop training method is ook dat ik geen encoder / states er voor hoef te initializen. kessels wel referencen. figuur gebruiken. * Jacobian additional states * 2d / 3d grid wat jan zei van de data space of ik alles vang van het gebied van telica. * Montecarlo simulatie zoals jan? * For parameter recovery I had trajectories similar to the paper, currently not used for joint estimation? * How well do we extrapolate outside multisine range? what does jan do?

Verdedigen

* waarom geen basis functions * waarom niet zoals kessels: vooral andere literatuur kunnen zeggen waarom niet dat gebruikt, moet dat? * wat is het voordeel van interpreteerbaarheid?–¿ controller maken, monitor health? * waarom niet frf of mass lijn fitten voor de missende dynamica * controller surpressses / hides issues such as orthogonality overlap. Byproduct of closed-loop. * Hoe verdedig ik augmentation over residual frf fit?? * Hoeveelheid data kunnen verdedigen * Huidige nf window and encoder window, should i run for different lengths? * Orthogonality definition and that we orthogonalize additional states with respect to the baseline := 0. additional states propogate, not included in our definition. * Kijken decisions.md * do i need to justify controller can i say what i got from my supervisor, how to approach these kind of questions?

References

[1] International Roadmap for Devices and Systems (IRDS), "International roadmap for devices and systems: 2022 edition executive summary," IEEE, Tech. Rep., 2022.

[2] T. Workman, L. Mirkarimi, J. Theil, G. Fountain, K. M. Bang, B. Lee, C. Uzoh, D. Suwito, G. Gao, and P. Mrozek, "Die to wafer hybrid bonding and fine pitch considerations," in *2021 IEEE 71st Electronic Components and Technology Conference (ECTC)*. IEEE, 2021, pp. 2071–2077.

[3] I. García-Herreros, X. Kestelyn, J. Gomand, R. Coleman, and P.-J. Barre, "Model-based decoupling control method for dual-drive gantry stages: A case study with experimental validations," *Control Engineering Practice*, vol. 21, no. 3, pp. 298–307, 2013.

[4] C. S. Teo, K. K. Tan, S. Y. Lim, S. Huang, and E. B. Tay, "Dynamic modeling and adaptive control of a h-type gantry stage," *Mechatronics*, vol. 17, no. 7, pp. 361–367, 2007.

[5] P. Lambrechts, M. Boerlage, and M. Steinbuch, "Trajectory planning and feedforward design for electromechanical motion systems," *Control Engineering Practice*, vol. 13, no. 2, pp. 145–157, 2005.

[6] J. Hwang and S. D. Noh, "Digital twin-based optimization of operational parameters for cluster tools in semiconductor manufacturing," *IEEE Access*, vol. 12, pp. 122 078–122 100, 2024.

[7] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating scientific knowledge with machine learning for engineering and environmental systems," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–37, Nov. 2022.

[8] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Learning-based augmentation of first-principle models: A linear fractional representation-based approach," 2026, submitted to Automatica.

[9] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, Jun. 2021.

[10] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations," *arXiv preprint arXiv:1711.10561*, 2017.

[11] B. M. Györök, M. Schoukens, T. Péni, and R. Tóth, "Orthogonal-by-construction augmentation of physics-based input-output models," *IFAC Journal of Systems and Control*, vol. 35, p. 100376, 2026.

[12] J. P. Noël and J. Schoukens, "Grey-box state-space identification of nonlinear mechanical vibrations," *International Journal of Control*, vol. 91, no. 5, pp. 1118–1139, 2018.

[13] B. M. Kessels, "Ai-based model updating for nonlinear dynamical systems," PhD thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2025.

[14] A. Retzler, R. Tóth, M. Schoukens, G. I. Beintema, J. Weigand, J.-P. Noël, Z. Kollár, and J. Swevers, "Learning-based augmentation of physics-based models: an industrial robot use case," *Data-Centric Engineering*, vol. 5, p. e12, 2024.

[15] R. Tóth, *Modeling and Identification of Linear Parameter-Varying Systems*, ser. Lecture Notes in Control and Information Sciences. Berlin, Heidelberg: Springer, 2010, vol. 403.

[16] R. Drenth, J. H. Hoekstra, M. Schoukens, and R. Tóth, "Efficient learning of affine and rational dependency LPV models with linear fractional representation," 2026.
[17] A. Ovchinnikov, A. Pillay, G. Pogudin, and T. Scanlon, "Computing all identifiable functions of parameters for ODE models," *Systems & Control Letters*, vol. 157, p. 105030, 2021.
[18] M. Gautier and W. Khalil, "Direct calculation of minimum set of inertial parameters of serial robots," *IEEE Transactions on Robotics and Automation*, vol. 6, no. 3, pp. 368–373, 1990.
[19] R. Drenth, "Efficient gradient-based learning of LPV models with linear fractional representation," Master's thesis, Eindhoven University of Technology, 2025.
[20] G. I. Beintema, M. Schoukens, and R. Tóth, "Deep subspace encoders for nonlinear system identification," *Automatica*, vol. 156, p. 111210, 2023.
[21] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Encoder initialisation methods in the model augmentation setting," 2026.
[22] U. Forssell and L. Ljung, "Closed-loop identification revisited: Updated version," Department of Electrical Engineering, Linköping University, Tech. Rep. LiTH-ISY-R-2021, Apr. 1998.
[23] A. Hefny, C. Downey, and G. J. Gordon, "Supervised learning for dynamical system learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
[24] C. Downey, A. Hefny, B. Li, B. Boots, and G. J. Gordon, "Predictive state recurrent neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
[25] A. Venkatraman, N. Rhinehart, W. Sun, L. Pinto, M. Hebert, B. Boots, K. M. Kitani, and J. A. Bagnell, "Predictive-state decoders: Encoding the future into recurrent networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
[26] B. M. Györök, J. H. Hoekstra, J. Kon, T. Péni, M. Schoukens, and R. Tóth, "Orthogonal projection-based regularization for efficient model augmentation," in *Proceedings of the 7th Annual Conference on Learning for Dynamics and Control*, ser. Proceedings of Machine Learning Research, N. Ozay, L. Balzano, D. Panagou, and A. Abate, Eds., vol. 283, 2025, pp. 166–178.
[27] C.-Y. Lee, S.-H. Hwang, E. Nam, and B.-K. Min, "Identification of mass and sliding friction parameters of machine tool feed drive using recursive least squares method," *The International Journal of Advanced Manufacturing Technology*, vol. 109, no. 9-12, pp. 2831–2844, 2020.

APPENDIX A
SUPPORTING DERIVATIONS

A. Explicit LPV-LFR realization

In (5), M_0 is (3) evaluated at $Y = 0$, and

$$M_1 = \begin{bmatrix} 0 & -m_h & 0 \\ -m_h & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad M_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & m_h & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

With a_1 and a_2 from (7) treated as independent signals, solving $M_0\ddot{q} + M_1 a_1 + M_2 a_2 = u - C\dot{q} - Kq$ for $\ddot{q}$ with $R = M_0^{-1}$ gives $\ddot{q} = R(u - Kq - C\dot{q} - M_1 a_1 - M_2 a_2)$. With $\dot{\tilde{x}} = \mathrm{col}(\dot{q}, \ddot{q})$, $z = \mathrm{col}(\ddot{q}, a_1)$, and $w = \mathrm{col}(a_1, a_2)$, the LTI part G of (10), partitioned as in [16, Eq. (6)], is

$$\begin{bmatrix} \dot{\tilde{x}} \\ z \\ q \end{bmatrix} = \underbrace{\begin{bmatrix} A_x & B_w & B_u \\ C_z & D_{zw} & D_{zu} \\ C_y & D_{yw} & D_{yu} \end{bmatrix}}_{G} \begin{bmatrix} \tilde{x} \\ w \\ u \end{bmatrix}. \tag{36}$$

with blocks

$$\begin{aligned} A_x &= \begin{bmatrix} 0 & I_3 \\ -RK & -RC \end{bmatrix}, & B_w &= \begin{bmatrix} 0 & 0 \\ -RM_1 & -RM_2 \end{bmatrix}, & B_u &= \begin{bmatrix} 0 \\ R \end{bmatrix}, \\ C_z &= \begin{bmatrix} -RK & -RC \\ 0 & 0 \end{bmatrix}, & D_{zw} &= \begin{bmatrix} -RM_1 & -RM_2 \\ I_3 & 0 \end{bmatrix}, & D_{zu} &= \begin{bmatrix} R \\ 0 \end{bmatrix}, \\ C_y &= \begin{bmatrix} I_3 & 0 \end{bmatrix}, & D_{yw} &= 0, & D_{yu} &= 0. \end{aligned} \tag{37}$$

Here $\tilde{x}, z, w \in \mathbb{R}^6$ and $u, q \in \mathbb{R}^3$, so $G \in \mathbb{R}^{15\times 15}$. The actuator frame enters only through $u = Pu_{\mathrm{act}}$ and $q_{\mathrm{act}} = P^\top q$ in (1).

Substituting $w = \Delta(Y)z$ into the z equation of (36) gives the algebraic loop equation $(I_6 - D_{zw}\Delta(Y))z = C_z x + D_{zu}u$. Written out,

$$\begin{bmatrix} I_3 + YRM_1 & YRM_2 \\ -YI_3 & I_3 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ a_1 \end{bmatrix} = \begin{bmatrix} R(u - Kq - C\dot{q}) \\ 0 \end{bmatrix}.$$

The second block row gives $a_1 = Y\ddot{q}$, and the first then gives $M(Y)\ddot{q} = u - C\dot{q} - Kq$, that is (2), after multiplication by M_0. The condition of [16, Def. 1], that $I_6 - D_{zw}\Delta(Y)$ is nonsingular, is therefore the condition that $M(Y)$ is nonsingular: the Schur complement of the lower identity block gives

$$\begin{aligned} \det\big(I_6 - D_{zw}\Delta(Y)\big) &= \det\big(I_3 + YRM_1 + Y^2RM_2\big) \\ &= \det M(Y)/\det M_0. \end{aligned}$$

Since $M(Y)$ is nonsingular, the loop is solved in closed form, $\ddot{q} = M(Y)^{-1}(u - C\dot{q} - Kq)$ with

$$M(Y)^{-1} = \frac{N_0 + YN_1 + Y^2N_2}{d(Y)}, \qquad d(Y) = \det M(Y),$$

where $N_0 + YN_1 + Y^2N_2 = \mathrm{adj}\, M(Y)$ and $d(Y)$ are both quadratic in Y. The implementation evaluates both polynomials with Horner's scheme. [**TODO:** Missing: print N_0, N_1, N_2 and $d(Y)$ explicitly? Candidate: no, appendix budget; the quadratic degree was checked symbolically (sympy, 2026-10-06).]

TABLE III: Nominal baseline parameters.

Parameter	Value	Parameter	Value
m_1	10.2 kg	m_2	10.7 kg
m_b	22.8 kg	m_h	10.1 kg
J_b	1.00 kg m^2	J_h	0.05 kg m^2
c_{g1}	14.5 N s/m	c_{g2}	20.3 N s/m
c_y	10.0 N s/m	d	0.10 m
c_{b1}	9.0 N m s/rad	c_{b2}	9.0 N m s/rad
k_{b1}	1987.5 N m/rad	k_{b2}	1987.5 N m/rad
L_b	0.725 m		

B. Identifiable combinations

For $m_h > 0$, the ten distinct entries of M_0, C and K determine θ_{base} of (11) uniquely (Section II-C):

$$\begin{aligned} &m_h = M_{0,33}, && d = -M_{0,23}/m_h, && m_\Sigma = M_{0,11} - m_h, \\ &m_\Delta = 2M_{0,12}/L_b, && J_{\mathrm{eff}} = M_{0,22} - m_h d^2, && k_{b,\Sigma} = K_{22}, \\ &c_{g1} = \tfrac{1}{2}C_{11} + C_{12}/L_b, && c_{g2} = \tfrac{1}{2}C_{11} - C_{12}/L_b, && c_y = C_{33}, \\ &c_{b,\Sigma} = C_{22} - \tfrac{L_b^2}{4}C_{11}. \end{aligned}$$

Conversely, (3) and (4) give every entry as a function of θ_{base}, so the map is one-to-one.

[**TODO:** Remarks cut from Section II-B; keep here or drop?]

- Y^2 is realized by repeating Y, not as a second scheduling variable $p_2 = Y^2$, which would admit pairs with $p_2 \neq Y^2$; extra scheduling variables cause overbounding in controller synthesis [16]
- The scheduling map is a known selection of the state ($Y = q_3$), not a learned map as in the self-scheduled models of [16]
- The exact condition $\det M(Y) \neq 0$ replaces the sufficient condition of [16, Thm. 6], which needs a scheduling set in the unit ball and $\rho(D_{zw}) < 1$
- Drenth's Def. 1 is stated in discrete time but constrains only the algebraic loop equation, so it applies unchanged in continuous time

C. Nominal physical parameters

Table III lists the nominal values used by the baseline. They are taken from the ETEL gantry reported by García-Herreros et al. [3, Table I].

D. Positive definiteness of the inertia matrix

With $a = L_b m_\Delta/2$, elimination of the m_h block and completion of the square give

$$\begin{aligned} \frac{\det M(Y)}{m_h} &= (m_\Sigma + m_h)(J_{\mathrm{eff}} + m_h Y^2) - (a - m_h Y)^2 \\ &= m_\Sigma m_h \left(Y + \frac{a}{m_\Sigma}\right)^2 \\ &\quad + (m_\Sigma + m_h)\left(J_{\mathrm{eff}} - \frac{a^2}{m_\Sigma}\right). \end{aligned}$$

The 2×2 Schur complement has positive leading entry $m_\Sigma + m_h$. Sylvester's criterion therefore reduces positive definiteness to positivity of the completed-square expression for every Y. Hence, for positive m_h, m_Σ, and J_{eff}, $M(Y) \succ 0$ for all $Y \in \mathbb{R}$

if and only if (12) holds. For the physical parameterization, the difference between the two sides is

$$m_\Sigma J_{\text{eff}} - \frac{L_b^2}{4} m_\Delta^2 = m_\Sigma (J_b + J_h) + \frac{L_b^2}{4} \left[4 m_1 m_2 + m_b (m_1 + m_2) \right],$$

which is strictly positive for positive component masses and yaw inertias.

In (24), let $b(\theta_{\text{base}}) = \rho (2/L_b) \sqrt{m_\Sigma J_{\text{eff}}}$. The offset and scale are

$$r_0 = \frac{m_{\Delta,0}}{b(\theta_{\text{base}}^0)}, \quad \eta_0 = \operatorname{atanh}(r_0), \quad s = \frac{m_{\Delta,0}}{b(\theta_{\text{base}}^0)(1 - r_0^2)}.$$

Here η_0 makes $\beta_\Delta = 0$ reproduce the initial combination, and s gives $\partial m_\Delta / \partial \beta_\Delta |_0 = m_{\Delta,0}$, matching the local relative-change scaling of the logarithmic coordinates. The construction requires $m_{\Delta,0} \neq 0$; a start near zero makes s small and the coordinate numerically inert. The exact inverse used for reporting is

$$\beta_\Delta = \frac{\operatorname{atanh}(m_\Delta / b(\theta_{\text{base}})) - \eta_0}{s}.$$

E. PS stopping rule

The unexplained fraction ν of (28) is evaluated after every i_cth update of V, with ν_i its value after update i. S1 ends at update

$$i_1 = \min \big\{ i \in \mathcal{J} : \; i = i_{\max} \text{ or } \big(i \geq i_{\min} \text{ and } \nu_i^{\text{new}} > (1 - \delta_\nu)\, \nu_i^{\text{old}} \big) \big\},$$

$$\nu_i^{\text{new}} = \min_{j \in \mathcal{W}_i} \nu_j, \qquad \nu_i^{\text{old}} = \min_{j \in \mathcal{J}_i \setminus \mathcal{W}_i} \nu_j,$$

where $\mathcal{J} = \{i_c, 2i_c, \dots\}$, $\mathcal{J}_i = \{j \in \mathcal{J} : j \leq i\}$, $\mathcal{W}_i$ the last w elements of $\mathcal{J}_i$, δ_ν the plateau tolerance and $i_{\min}$, $i_{\max}$ the earliest and latest end of S1, with $\nu_i^{\text{old}} = \infty$ while $\mathcal{J}_i$ has at most w elements. S2 runs if $\nu_{i_1} \leq \nu_{\max}$ and, when $i_1 > i_s$, also $\nu_{i_s} \leq \nu_{\max}$, where $i_s < i_{\max}$ is a fixed early check. If $\nu_{i_s} > \nu_{\max}$, PS ends at i_s. The values are given in Section V.

F. OBC and experiment details

- Derive the stacked projector, rank-deficient pseudoinverse case, epoch refresh, and additional-state zero extension
- Derive $J^+ \Delta^*$ and list the measured singular-direction contributions to the predicted parameter bias
- List dataset identifiers, split membership, hyperparameters, seed, and exact metric definitions for every reported arm
- Figure (existing, optional): place `controller-chain-v1` here only if the controller reconstruction cannot be audited adequately in prose and equations

[**TODO:** Can code and data tables be cited through a repository or DOI instead of spending paper space on full reproducibility metadata?]